The University of British Columbia - Okanagan



Project Name: Canvas AI Assistant

Course: COSC 499 – Capstone

Project Client: Dr. Scott Fazackerley

Team 16: Jessica Goel, Mridul Nohria, Bao Tran, Yerdana Maulenbay, Rhythm Trivedi

TABLE OF CONTENTS

Section	Page
Cover Page	1
Table of Contents	2
1. Overview	3
2. System Architecture	7
3. System Features	14
4. Status of System Features	28
5. UX Analysis	36
6. Project Management Activities	46
7. Lessons Learned	55
8. Installation & User Guide	62
9. Conclusion	74

Total pages: 75

1. Overview

1.1 Project Overview

Project Name

Canvas AI Assistant for Course Management

Course

COSC 499 – Capstone Software Engineering Project

Client

Dr. Scott Fazackerley

Team Members

- Jessica Goel-Frontend developer
- Mridul Nohria-Integration Lead / DevOps / QA Lead
- Bao-Backend developer
- Rhythm-Frontend developer
- Yerdana-Backend developer

1.2 Project Background

Learning Management Systems (LMS) such as Canvas provide instructors with a wide range of tools for managing courses, assignments, grading, and communication. Although these systems are feature-rich, many instructor workflows remain repetitive and time-consuming. Tasks such as identifying students with missing submissions, composing reminder messages, updating assignment details, and navigating large amounts of course information require significant manual effort.

Recent advances in Large Language Models (LLMs) present an opportunity to automate portions of these workflows. However, directly integrating AI into educational systems raises concerns regarding privacy, reliability, transparency, and user trust. Institutions require AI systems that maintain instructor control while ensuring that sensitive educational data is handled responsibly.

The Canvas AI Assistant addresses these challenges by providing an AI-assisted workflow that supports instructors while maintaining a human-in-the-loop approval process. Rather than allowing AI to directly modify Canvas content, the system generates suggestions that instructors review before execution. This approach improves efficiency without sacrificing accuracy or accountability.

1.3 Problem Statement

Canvas provides comprehensive functionality for course management but lacks intelligent workflow assistance for common instructor tasks. Activities such as locating students with missing submissions,

drafting reminder messages, creating assignments, and updating assignment information require multiple manual steps across different interfaces.

These repetitive tasks increase instructor workload and reduce the time available for teaching and student engagement. Existing AI solutions often prioritize automation but may not provide sufficient transparency or control, making them unsuitable for educational environments where instructor oversight is essential.

1.4 Proposed Solution

The project delivers a web-based AI assistant that integrates with the Canvas Learning Management System to support instructors in performing common administrative tasks more efficiently.

The application combines a React frontend with a FastAPI backend and integrates Large Language Models through an externally hosted API, using the backend emulating MCP-like functionality. Model Context Protocol (MCP) serves as a separate server for users with existing MCP Clients such as Codex, Claude Code; however, full functionality is available on a native MCP-like backend. The AI analyzes course information and generates suggestions for instructor review, while all high-impact actions require explicit instructor approval before being executed.

The system follows a human-in-the-loop design philosophy, ensuring that instructors remain fully responsible for all changes made within Canvas.

1.5 Unique Value Proposition

Unlike many AI-powered educational tools that emphasize full automation, the Canvas AI Assistant focuses on responsible AI adoption within higher education.

The system offers:

- Human approval before every high-impact action.

Privacy-focused backend infrastructure that gatekeeps PII from reaching externally hosted LLM.

- Seamless integration with existing Canvas workflows.
- AI-assisted assignment creation and modification.
- Intelligent drafting of student reminder messages.
- Administrative tools for managing instructor accounts.
- Transparent previews of AI-generated content before execution.

This combination of AI assistance and instructor control provides increased productivity while maintaining institutional trust and accountability.

1.6 User Groups

The system supports two primary user groups.

Instructor

Instructors use the system to connect their Canvas accounts, retrieve course information, identify students with missing submissions, generate reminder messages, create and update assignments, preview AI-generated actions, and approve changes before they are applied to Canvas.

Administrator

Administrators manage instructor accounts, assign user roles, enable or disable access to the system, and oversee platform administration. Administrative users do not perform instructional workflows but ensure that the system remains secure and properly managed.

1.7 Functional Requirements

The completed system supports the following major functional requirements:

- User authentication
- Role-based access control
- Instructor account management
- Canvas integration
- Course retrieval
- Assignment retrieval
- Student retrieval
- Missing submission analysis
- AI-generated reminder messages
- Assignment creation
- Assignment updating
- Dry-run preview workflow
- Human approval before execution
- Administrative user management
- Audit logging

AI Provider integration through externally hosted LLM reachable via API

Additional functionality implemented during development includes improved confirmation dialogs, enhanced validation, structured assignment editing forms, and expanded testing support.

1.8 Non-Functional Requirements

The project satisfies several important non-functional requirements.

Security

- JWT authentication
- Password hashing
- Role-based authorization
- Secure API communication

Reliability

- Human approval before execution
- Validation of AI-generated responses
- Backend error handling
- Logging of important operations

Usability

- Modern responsive interface
- Accessible navigation
- Dashboard-based workflow
- Consistent user interface

Performance

Externally hosted LLM with API

-
- Efficient API communication
- Optimized frontend rendering

Maintainability

- 3 Layer Monolithic architecture for backend
- Component-based frontend
- Layered backend services
- Automated testing

1.9 Technical Requirements

The application was implemented using the following technology stack:

Layer	Technology
Frontend	React, TypeScript, Tailwind CSS, shadcn/ui
Backend	FastAPI, Python, Pydantic

AI	Huggingface API, MCP Like Dispatcher
Database	PostgreSQL
Authentication	JWT
Version Control	GitHub
CI/CD	GitHub Actions
Containerization	Docker
Testing	Pytest, Vitest, Pydantic validation

1.10 Scope Summary

The final system successfully implements the project's core objectives by combining AI assistance with instructor oversight. The completed software enables instructors to streamline repetitive Canvas workflows while ensuring that all AI-generated actions remain transparent, reviewable, and under human control. Throughout development, additional usability improvements identified during user testing were incorporated into the system, resulting in a more intuitive and reliable user experience.

2. System Architecture

2.1 Architecture Overview

The Canvas AI Assistant follows a modular client-server architecture designed to separate presentation, business logic, artificial intelligence processing, and data persistence into independent layers. This separation improves maintainability, scalability, testing, and future extensibility while allowing each component to evolve independently.

The system consists of a React-based frontend, a FastAPI backend, a PostgreSQL database, an externally hosted Large Language Model (LLM) reachable via API, an MCP-like dispatcher integrated into the backend, an optional separate MCP server for existing MCP clients (Codex or Claude Code), and integration with the Canvas Learning Management System through

the Canvas REST API. Communication between components occurs using REST APIs and JSON data.

Unlike traditional AI systems that allow direct execution of generated content, the Canvas AI Assistant follows a **human-in-the-loop** architecture. All AI-generated outputs are presented to the instructor for review before any modifications are made to Canvas. This design ensures transparency, maintains instructor control, and reduces the risk of unintended actions.

The architecture also emphasizes modularity by separating authentication, Canvas integration, AI services, and administrative functionality into independent backend modules. This organization improves maintainability and allows future extensions, such as support for additional AI providers, without requiring major architectural changes.

2.2 Overall System Architecture

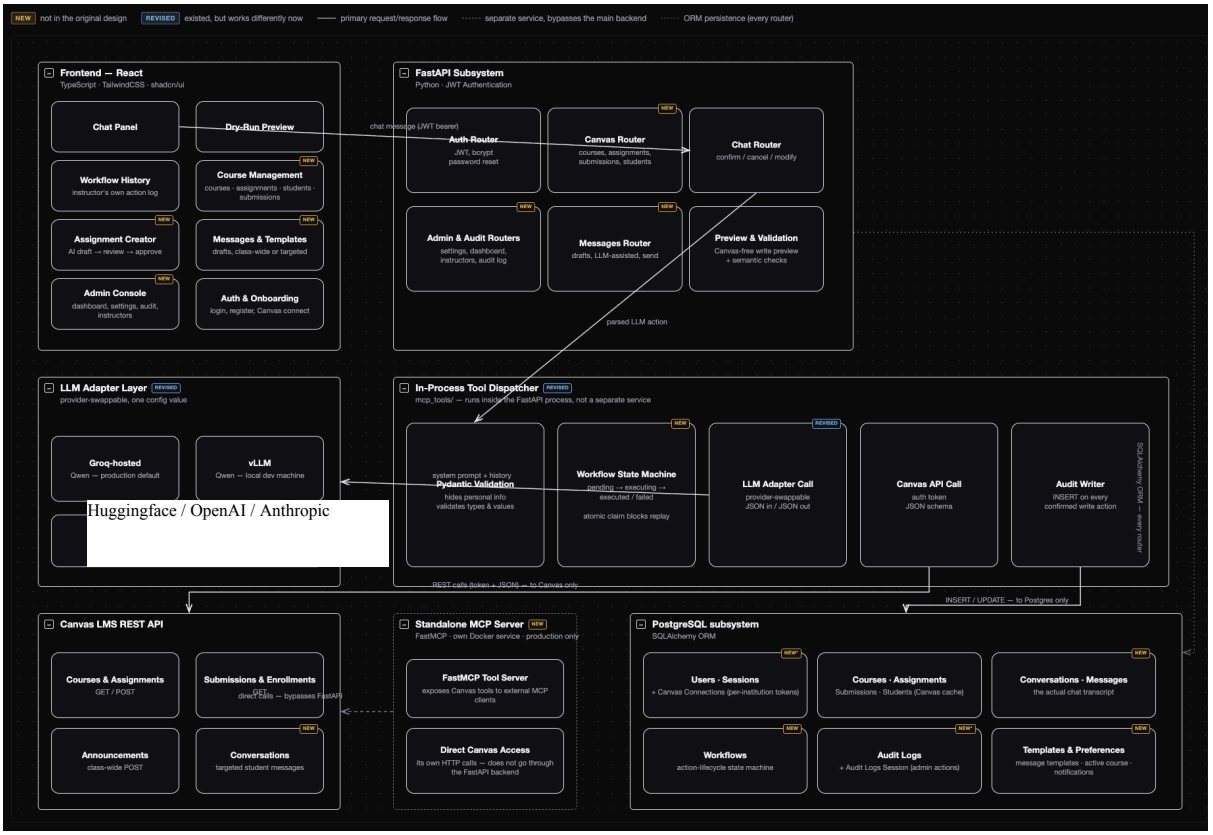


Figure 2.1. Final System Architecture

Explanation of the System Architecture

Figure 2.1 illustrates the overall architecture of the Canvas AI Assistant and the interactions between its major components.

The workflow begins when an instructor accesses the application through a web browser. The React frontend provides the user interface for authentication, course selection, assignment management, AI interactions, and administrative functions.

When a user performs an action, such as requesting missing submissions or generating reminder messages, the frontend sends an authenticated HTTP request to the FastAPI backend. The backend validates the user's JWT token, verifies permissions using Role-Based Access Control (RBAC), and determines the appropriate service responsible for processing the request.

Depending on the requested operation, the backend communicates with one or more external components.

For Canvas-related operations, the backend interacts with the Canvas REST API using the instructor's stored Canvas credentials. Course information, assignments, submissions, and student records are retrieved directly from Canvas and converted into standardized internal data models before being returned to the frontend.

For AI-assisted operations, the backend incorporates a set of tools defined that emulates MCP like behaviour. These tools defined under dispatcher serve the Large Language Model input and output by validating fields, values, and by constraining PII from entering LLM. Prompts are transformed into JSON structures that contain schemas defined in our backend, thus every action must adhere to available schemas. Every prompt gets concatenated with the system prompt, and available chained commands to enhance the context that goes into LLM.

The PostgreSQL database stores application data, including user accounts, encrypted authentication information, audit logs, application settings, and other persistent system data. The database does not store Canvas course content beyond what is required for application functionality, minimizing unnecessary duplication of institutional data.

Once the backend completes processing, the response is returned to the frontend, where instructors can review generated content before approving or rejecting any action. Only after explicit user approval does the backend execute modifications within Canvas.

This workflow ensures that AI remains an assistive technology rather than an autonomous decision-maker.

2.3 Technology Stack

The project was implemented using modern web development technologies selected based on reliability, community support, ease of maintenance, and compatibility with AI-enabled applications.

Layer	Technology	Purpose	Justification
Frontend	React	User Interface	Component-based architecture enables reusable, maintainable UI development.
Language	TypeScript	Type Safety	Reduces runtime errors and improves code maintainability.
Styling	Tailwind CSS	UI Styling	Rapid development with consistent responsive design.
UI Components	shadcn/ui	Reusable Components	Provides accessible, customizable UI components.
Backend	FastAPI	REST API	High performance, automatic API documentation, and Python ecosystem support.
Language	Python	Backend Logic	Extensive AI and web framework support.

AI Runtime	Huggingface hosted LLM	Externally hosted LLM with API	Privacy-focused backend gatekeeping of PII with externally hosted LLM via API.
AI Integration	MCP Like dispatcher	Tool Orchestration	MCP-like dispatcher integrated into the backend for structured LLM tool calls.
Database	PostgreSQL	Persistent Storage	Reliable relational database supporting structured application data.
Authentication	JWT	User Authentication	Stateless authentication suitable for REST APIs.
Containerization	Docker	Deployment	Consistent execution environment across development and testing.
CI/CD	GitHub Actions	Continuous Integration	Automated testing and code quality validation.
Backend Testing	Pytest, Pydantic validation	n Testing	Reliable testing framework for Python services.
Frontend Testing	Vitest	Frontend Testing	Fast testing for React components and business logic.

2.4 Frontend Architecture

The frontend was developed using React with TypeScript following a component-based architecture. User interface elements are organized into reusable components that separate presentation from business logic, improving maintainability and reducing code duplication.

The frontend is responsible for:

- User authentication
- Dashboard navigation
- Canvas course selection
- Assignment management
- AI interaction interface
- Preview workflows
- Administrative interfaces
- Displaying API responses
- Input validation

Communication with the backend occurs through REST API requests using JSON payloads. Authentication tokens are included with each request to ensure secure access to protected resources.

2.5 Backend Architecture

The backend follows a layered architecture implemented using FastAPI.

Major backend layers include:

API Layer

Receives HTTP requests from the frontend and returns JSON responses.

Responsibilities:

- Route requests
- Validate input
- Return responses
- Handle errors

Service Layer

Contains the business logic.

Responsibilities include:

- Authentication
- Canvas communication
- Assignment workflows
- AI orchestration
- User management
- Audit logging

Separating business logic from API controllers improves testability and allows services to be reused across multiple endpoints.

Data Access Layer

Responsible for interacting with PostgreSQL.

Responsibilities include:

- User persistence
- Role management
- Settings
- Audit logs
- Authentication information

2.6 AI Architecture

One of the defining features of the project is its AI integration. Instead of allowing AI models to communicate directly with Canvas, the system introduces an intermediate MCP-like dispatcher integrated into the backend, which orchestrates calls to an externally hosted Hugging Face LLM via API.

The AI workflow consists of the following steps:
1. The instructor initiates an AI-assisted task.
2. The backend gathers the necessary contextual information from Canvas.
3. The information is formatted into a structured prompt and concatenated with the system prompt and available chained commands.
4. The MCP-like dispatcher validates fields and values and constrains PII from entering the LLM.
5. The prompt is sent to the externally hosted Hugging Face LLM via API.
6. The language model generates a structured response that must adhere to backend-defined schemas.
7. The backend validates the AI output.
8. The frontend presents the result for instructor review.
9. The instructor either approves or rejects the action.
10. Approved actions are executed against the Canvas API.

A separate Model Context Protocol (MCP) server remains available for users with existing MCP clients such as Codex or Claude Code; however, full in-app functionality is provided by the native MCP-like backend dispatcher. This architecture ensures that AI-generated content never bypasses instructor approval.

2.7 Authentication and Security

Security is implemented through several complementary mechanisms.

Authentication uses JSON Web Tokens (JWT) to identify authenticated users after login.

Passwords are securely hashed before storage and never stored in plain text.

Role-Based Access Control (RBAC) restricts administrative functionality to authorized users while preventing unauthorized access to instructor-specific operations.

Input validation is performed at both the frontend and backend to reduce the risk of malformed requests.

Audit logs record significant user actions to improve traceability and accountability. Encryption of Instructor Canvas API Tokens using Fernet encryption protects stored Canvas credentials at rest.

2.8 Data Flow Diagram – Level 0

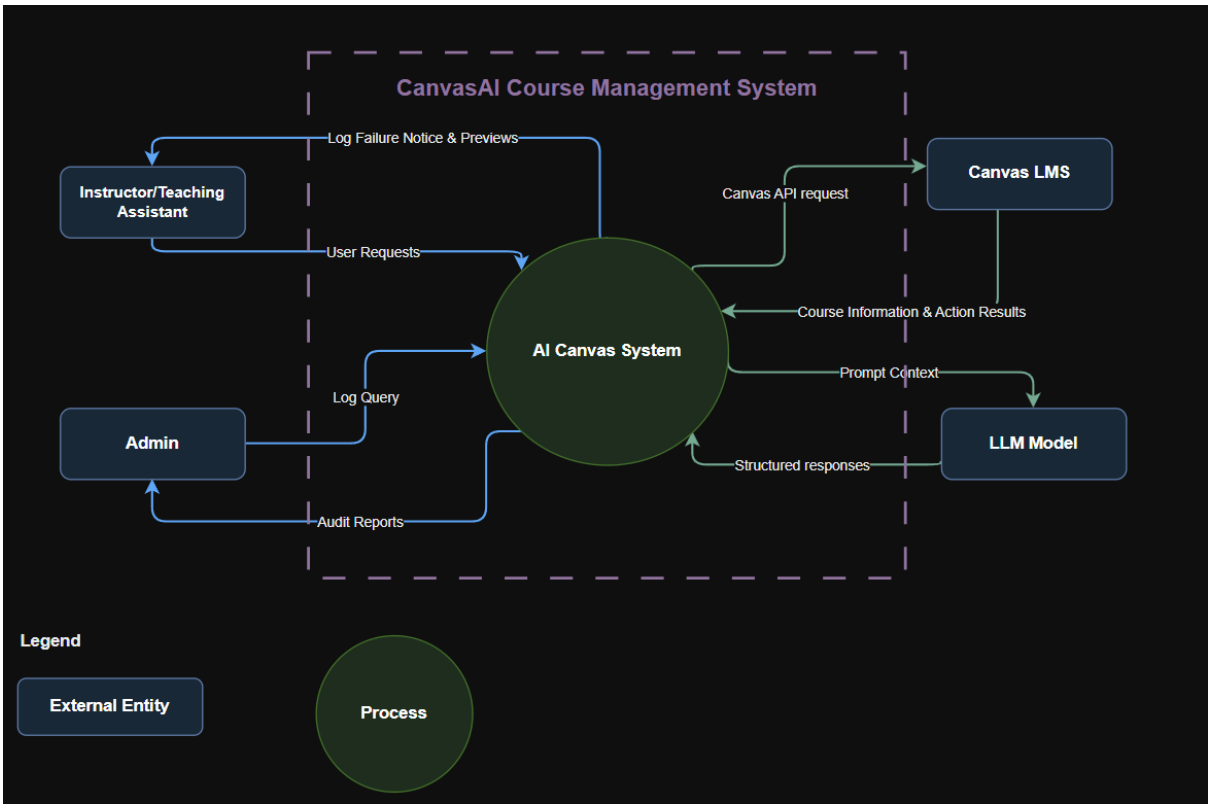


Figure 2.2. Level 0 Data Flow Diagram

Explanation

The Level 0 Data Flow Diagram presents the system as a single process interacting with external entities.

The primary external entities are:

- Instructor
- Administrator
- Canvas LMS

The instructor and administrator submit requests to the system through the web interface. The system processes these requests, retrieves or updates information in Canvas when required, stores application-specific information within PostgreSQL, and returns the resulting information to the user.

The Level 0 diagram provides a high-level overview of information movement without exposing internal implementation details.

2.9 Data Flow Diagram – Level 1

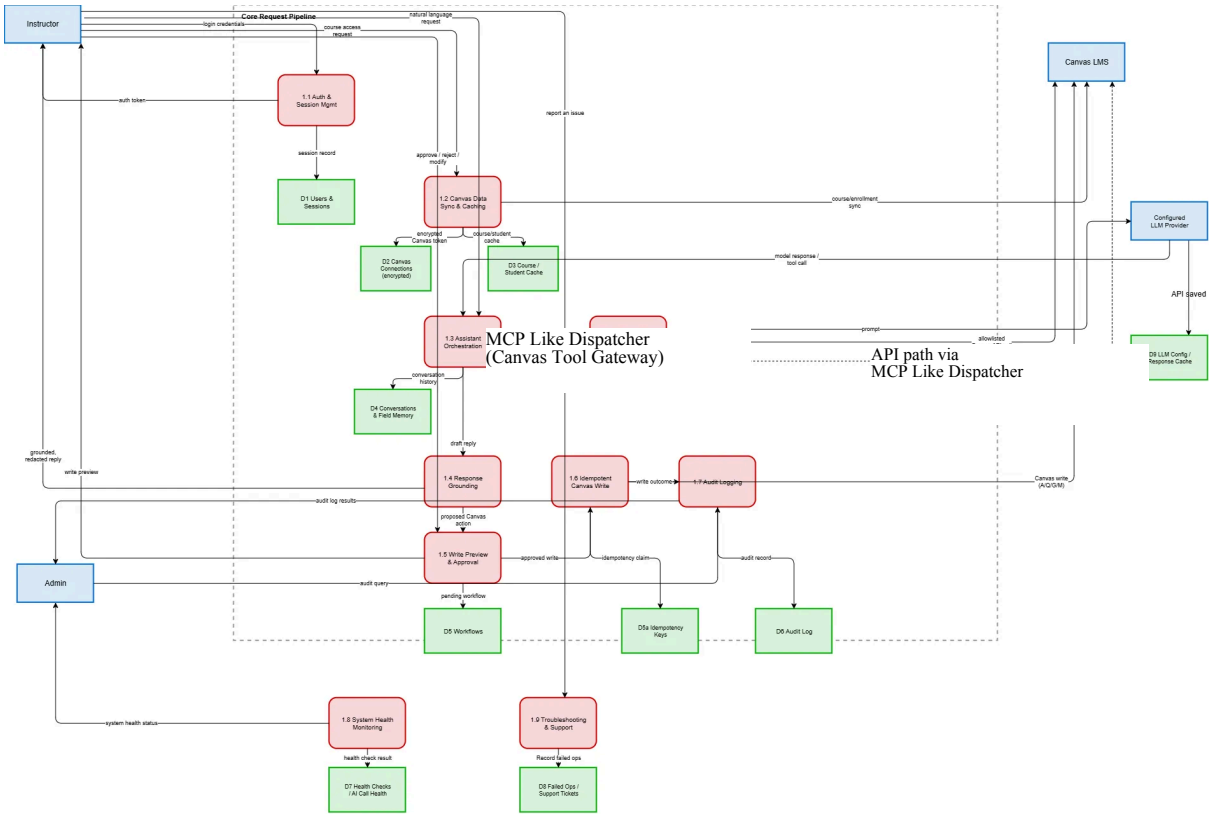


Figure 2.3. Level 1 Data Flow Diagram

Explanation

The Level 1 DFD expands the internal processes represented in the Level 0 diagram. AI mediation uses the MCP Like Dispatcher in the backend.

Major processes include:

- User Authentication
- Course Retrieval
- Assignment Retrieval
- Student Retrieval
- Missing Submission Analysis
- AI Prompt Generation
- Reminder Draft Generation
- Assignment Creation
- Assignment Update
- Preview Approval Workflow
- Administrative Management

- Audit Logging

Each process communicates with the appropriate data stores and external systems while exchanging structured information with the frontend.

The Level 1 diagram illustrates how requests move through multiple processing stages before a final response is returned to the instructor.

3. System Features

3.1 System Feature Overview

The Canvas Helper AI Assistant was developed as a web-based system that assists instructors with common Canvas LMS course-management activities while maintaining administrator oversight and explicit human control over Canvas-modifying actions. The final system combines secure user authentication, Canvas integration, course and submission analysis, natural-language interaction, assignment and quiz management, messaging and feedback workflows, grade modification, action history, administrative monitoring, AI validation, and security safeguards within a single application.

A defining characteristic of the final system is its **confirm-before-write** approach. Actions that modify Canvas are not executed immediately after being generated by the assistant. Instead, proposed actions are interpreted, validated, presented to the instructor as a preview, and executed only after explicit confirmation. The final architecture also treats Canvas as the source of truth for course information while the local application database maintains workflow state, audit information, and user preferences.

The initial project requirements defined two primary user groups: administrators, who manage accounts, configuration, monitoring, audit information and operational health, and instructors, who use the dashboard and natural-language assistant to manage Canvas workflows.

All user and functional requirements included in the final Testing Event 2 report were recorded as **PASS**. Testing included combinations of unit, integration, system functionality, operational acceptance, usability and acceptance testing depending on the requirement being evaluated.

3.2 Authentication and Account Management

Authentication provides the security boundary for the entire application. Users must authenticate before accessing protected application functionality, particularly functionality that retrieves or modifies Canvas information.

The system supports both instructor and administrator access. Instructor accounts can be created and managed through administrative functionality, while disabled or revoked accounts are prevented from continuing to access the application.

The authentication and account-management requirements include secure login, creation and management of instructor accounts, revocation of access, Canvas token management, session handling, and protection of credentials. The original functional requirements further specify that Canvas functionality must not be accessible until authentication succeeds.

The final system provides:

- authenticated login and logout;

- account creation and access control;
- enable/disable functionality for instructor accounts;
- revocation of instructor access;
- session management;
- authorization of protected operations; and
- separation of instructor and administrator functionality.

Authentication and account-management requirements passed TE2 testing. Contributors identified in the test report for these requirements include **Jessica, Mridul and Bao**, with Jessica also contributing to application account authentication alongside Mridul and Yerdana.

Related requirements: UR1.1–UR1.3, UR2.1, FR1.1–FR1.3, FR1.10
Final status: Fully implemented and verified – PASS

3.3 Canvas Connection and Credential Management

The Canvas connection feature allows an authenticated instructor to establish the connection between the Canvas Helper AI Assistant and Canvas LMS.

Canvas API credentials are treated as sensitive data. The system supports entering, replacing and removing Canvas credentials while protecting them from exposure after submission. The requirements specify that Canvas API tokens must be encrypted before storage, validated before use and isolated so that one instructor cannot retrieve another instructor's credential.

This feature supports:

- securely connecting a Canvas API token;
- validating the token before use;
- replacing an existing token;
- removing a stored token;
- encrypted credential storage;
- preventing display of previously stored credentials; and
- ensuring Canvas operations use the authenticated instructor's connection.

The final architecture further specifies that encrypted Canvas tokens are not provided to the LLM.

TE2 recorded Canvas connection, replacement/removal, token validation, encrypted storage and token isolation as passing requirements. Jessica, Mridul and Bao contributed to the connection workflows, with Mridul and Bao identified on several of the backend token-management requirements.

Related requirements: UR2.2–UR2.3, FR1.4–FR1.9
Final status: Fully implemented and verified – PASS

3.4 Course Dashboard and Canvas Data Access

The instructor dashboard serves as the primary entry point for course-management workflows. After successfully connecting Canvas, the instructor can retrieve available courses, select an active course and access information relating to that course.

Canvas is treated as the authoritative source for course and assignment information rather than duplicating this information as an independent local source.

The implemented dashboard and Canvas data-access functionality includes:

- retrieving courses associated with the connected instructor;
- selecting an active course;
- retrieving students;
- retrieving assignments;
- retrieving submissions;
- displaying course-level summaries;
- displaying assignment and submission information;
- identifying submitted, missing, late and graded submissions;
- searching and filtering course information; and
- providing suggested workflow actions and shortcuts.

The selected active course is also used to scope assistant requests unless the instructor explicitly identifies another course. This reduces ambiguity and maintains context as the instructor moves through application workflows.

TE2 verified all course dashboard and Canvas data-access requirements as passing. Contributors identified for these features include **Mridul, Rhythm and Bao**, with **Jessica also contributing to student retrieval and course-level dashboard information**.

Related requirements: UR2.4–UR2.10, UR2.17, FR2.1–FR2.10
Final status: Fully implemented and verified – PASS

3.5 Natural-Language Assistant

The Natural-Language Assistant provides a conversational interface through which instructors can request Canvas-related information and initiate supported course-management workflows without manually navigating every individual function.

The assistant interprets natural-language requests and converts them into structured actions. When a request is ambiguous or incomplete, the assistant is expected to request clarification rather than make unsafe assumptions. Before an executable action is carried out, the interpreted intent is made available to the instructor.

The implemented assistant supports operations including:

- listing courses;
- listing students;
- listing assignments;
- listing submissions;
- creating assignments;
- updating assignment information and dates;
- drafting student messages;
- drafting feedback;
- creating quizzes; and
- initiating grade-modification workflows.

The assistant also prevents unsupported actions from being executed and must not invent Canvas information that has not actually been retrieved from Canvas.

The final repository identifies the natural-language assistant and preview workflow as a major implemented feature, with backend routing and workflow functionality forming part of its implementation.

TE2 records the Natural-Language Assistant requirements as passing and identifies **Jessica, Mridul and Yerdana** as contributors.

Related requirements: UR2.11–UR2.13, FR3.1–FR3.9
Final status: Fully implemented and verified – PASS

3.6 Submission Analysis

Submission analysis allows instructors to understand assignment submission status without manually inspecting each student individually.

For a selected assignment, the system can identify students whose submissions are missing or late and can present submission summaries and affected student lists. The course dashboard can additionally represent submitted and graded states where the required information is available from Canvas.

This feature provides:

- assignment submission retrieval;
- missing-submission identification;
- late-submission identification;
- affected-student lists;
- submission-status summaries; and
- integration with follow-up workflows such as messaging and grading.

TE2 records the submission-analysis requirements as passing. Contributors listed for this area are **Mridul, Rhythm and Bao**.

Related requirements: UR2.7, UR2.8, UR2.17, FR5.1–FR5.3
Final status: Fully implemented and verified – PASS

3.7 Assignment Management

The assignment-management feature allows instructors to create new Canvas assignments and modify existing ones through controlled workflows.

For assignment creation, the instructor provides the necessary information and the application prepares the assignment for validation and preview before it reaches Canvas. Existing assignments can similarly be retrieved and updated, including modification of assignment dates.

The requirements explicitly require assignment data to be validated before submission and assignment creation or modification to pass through preview and confirmation.

The implemented feature therefore includes:

- creation of Canvas assignments;
- retrieval of existing assignment information;
- editing assignment details;
- updating assignment dates;
- validation of proposed assignment data;
- preview of proposed changes;
- instructor confirmation; and
- execution through the approved Canvas workflow.

TE2 reports assignment creation, assignment updating, validation and preview/confirmation as passing. **Jessica, Mridul and Yerdana** are identified as contributors.

Related requirements: UR2.14–UR2.15, FR4.1–FR4.2, FR4.4–FR4.5
Final status: Fully implemented and verified – PASS

3.8 Quiz Creation

The final system extends content-management functionality beyond assignments by supporting creation of Canvas quizzes.

Quiz information is handled using the same safety principles applied to other Canvas-modifying workflows. Information must be validated before submission, and the user receives a preview before explicitly confirming creation.

Quiz creation was present in the original requirements under UR2.16 and FR4.3.

TE2 confirms that quiz creation, validation and the associated confirmation workflow passed testing. The user-requirement-level report specifically lists **Jessica** as the contributor for quiz creation, while the functional feature was tested alongside the assignment and quiz management workflow involving Jessica, Mridul and Yerdana.

Related requirements: UR2.16, FR4.3–FR4.5
Final status: Fully implemented and verified – PASS

3.9 Canvas Messaging

Canvas Messaging enables instructors to draft messages to selected students through the assistant rather than manually composing each communication.

The generated message is treated as a draft rather than immediately sent. The instructor can review and edit the message before delivery. Sending the message is considered a Canvas-modifying action and therefore passes through preview and confirmation.

The requirements define message generation, editing, preview/confirmation and history recording as part of this workflow.

Implemented functionality includes:

- generation of draft Canvas messages;
- selection of appropriate students;
- review of generated content;
- editing before delivery;
- preview before execution;
- explicit confirmation; and
- recording the message workflow in action history.

TE2 records all messaging requirements as passing, with **Jessica, Mridul and Yerdana** identified as contributors.

Related requirements: UR2.18–UR2.19, FR6.1–FR6.3, FR6.6
Final status: Fully implemented and verified – PASS

3.10 Feedback Generation and Posting

The feedback workflow assists instructors in producing text feedback for selected student submissions.

As with messaging, generated feedback is not automatically posted. The instructor can review the proposed comment and approve it before the backend performs the corresponding Canvas operation.

The original requirements specify both drafting feedback and requiring review and approval before posting.

The final implementation provides:

- feedback-comment drafting;
- association with selected submissions;
- preview of generated feedback;
- instructor review;
- explicit confirmation before posting; and
- history recording.

TE2 verifies the feedback requirements as passing. At the user-requirement level, **Jessica** is listed as the contributor for drafting and approving feedback. The wider messaging/feedback functional workflow lists Jessica, Mridul and Yerdana.

Related requirements: UR2.20–UR2.21, FR6.4–FR6.6
Final status: Fully implemented and verified – PASS

3.11 Grade Modification

The grade-modification workflow allows instructors to modify grades through an approved Canvas workflow.

Because grading is a high-impact action, grade changes are not sent directly to Canvas. A proposed grade modification is prepared and validated, presented through the preview system, and executed only after confirmation.

The original functional requirements explicitly require both grade modification and preview/confirmation for grade changes.

The implemented grade workflow includes:

- identification of the relevant submission;
- preparation of a grade modification;
- validation of the proposed operation;
- preview of the change;
- explicit user confirmation; and
- controlled execution against Canvas.

TE2 reports both grade-modification requirements as passing and identifies **Jessica and Mridul** as contributors to the tested grade workflow.

Related requirements: UR2.22, FR5.4–FR5.5
Final status: Fully implemented and verified – PASS

3.12 Dry-Run Preview and Confirm-Before-Write Workflow

The Dry-Run Preview is one of the central safety mechanisms of the Canvas Helper AI Assistant.

Every action that modifies Canvas first produces a preview describing the intended action and the affected Canvas resources. The preview communicates whether the operation can be reversed and allows the instructor either to confirm or cancel the action.

The system further classifies actions according to risk, including read-only, modifying and irreversible actions.

The final repository explicitly identifies **confirm-before-write for all Canvas mutations** as a core architectural decision.

The workflow can be summarized as:

Instructor Request → Intent Interpretation → Backend Validation → Proposed Action → Dry-Run Preview → Instructor Confirmation/Cancel → Canvas Execution → History/Audit Recording

This approach prevents the AI model from acting autonomously on Canvas and ensures that the instructor remains the final decision-maker.

TE2 reports all dry-run and confirmation requirements as passing. Contributors include **Jessica, Mridul and Yerdana**, with Rhythm also contributing to approval and cancellation requirements at the user-requirement level.

Related requirements: UR2.13, UR2.23–UR2.25, FR7.1–FR7.6
Final status: Fully implemented and verified – PASS

3.13 Action History and Limited Reversal

The application maintains a user-visible history of workflows and actions to improve transparency and accountability.

History records include information describing the original request, the action performed, execution status, timestamp and affected Canvas resources. Assistant requests can therefore be connected to their resulting actions and outcomes.

Where supported and safe, the system also provides limited reversal functionality. When an action cannot safely be reversed, the application communicates that limitation rather than presenting reversal as universally available.

The implemented feature includes:

- user-visible workflow history;
- assistant request history;
- execution status;
- timestamps;
- affected-resource information;
- reversibility information;
- limited supported reversal; and
- explanation when reversal is unavailable.

TE2 reports the history and reversal functionality as passing. The feature involved **Mridul, Rhythm, Bao and Yerdana**, with Jessica also listed for assistant request/action history.

Related requirements: UR2.26–UR2.28, FR8.1–FR8.5
Final status: Fully implemented and verified – PASS

3.14 Administrator Monitoring, Audit and Troubleshooting

Administrators require visibility into the health and operation of the system in addition to account-management functionality.

The administrative functionality therefore includes operational health information, audit logs, failed Canvas operations, failed AI operations and troubleshooting support that does not require exposing sensitive credentials.

The project requirements define administrators as responsible for account management, system configuration, security settings, monitoring, audit logs, deployment infrastructure and operational health.

Implemented administrative features include:

- instructor account management;
- enable/disable and access revocation;
- audit-log review;
- security-event review;
- operational-health information;
- monitoring failed Canvas operations;
- monitoring failed AI operations; and
- troubleshooting support.

TE2 records system health, audit-log access, failure monitoring and troubleshooting as passing. Bao is identified as a primary contributor for health, audit and troubleshooting functionality, while Rhythm contributed to failed-operation functionality.

Related requirements: UR1.2–UR1.7, FR12.1–FR12.2
Final status: Fully implemented and verified – PASS

3.15 AI Validation and MCP/Tool Integration

The AI model is deliberately separated from direct Canvas execution.

Instead, operations are routed through backend validation and, where used, the integrated MCP Like dispatcher in backend layer. The backend rejects malformed, unsupported, ambiguous, unauthorized or unsafe actions before execution.

tool layer is restricted to approved Canvas operations.

The final architecture describes MCP as an **optional credential/tool boundary**, while FastAPI remains responsible for application services and controlled Canvas interaction.

The final repository also documents support for an LLM-provider layer rather than coupling the system to only one model provider.

This feature provides:

- structured AI-generated actions;
- backend validation;
- allowlisted Canvas operations;
- rejection of unsupported actions;
- restricted credential handling;
- optional MCP/tool-server mediation; and
- support for LLM provider integration.

TE2 reports all AI validation and MCP integration requirements as passing, with **Jessica, Mridul and Rhythm** identified as contributors.

Related requirements: FR9.1–FR9.6
Final status: Fully implemented and verified – PASS

3.16 Security, Privacy and Reliability

Security and privacy were treated as system-wide requirements rather than individual UI features.

Sensitive Canvas credentials and AI credentials must be encrypted at rest and prevented from appearing in application logs, errors, API responses, repositories or the user interface. Student data is limited to authorized users, and student-identifiable information sent to AI services is minimized.

Reliability requirements further require meaningful user-facing errors, safe handling of Canvas API failures, safe handling of unavailable Canvas data and prevention of duplicate execution of Canvas-modifying actions.

Implemented safeguards therefore include:

- credential encryption;
- authorized access to student data;
- AI-data minimization;
- backend validation of mutations;
- meaningful error responses;
- handling Canvas API failures;
- handling unavailable or incomplete Canvas data; and
- duplicate-write protection.

The final architecture reinforces these protections by stating that Canvas tokens are encrypted and never sent to the LLM.

TE2 reports all security/privacy and reliability requirements as passing. **Jessica, Rhythm and Bao** contributed to security/privacy requirements, while **Jessica and Rhythm** are identified on reliability and error handling.

Related requirements: UR2.29, FR10.1–FR10.5, FR11.1–FR11.4
Final status: Fully implemented and verified – PASS

3.17 Deployment and Operational Support

The project is implemented as a web-based application and includes Docker-based deployment support.

The final repository provides commands for both development/testing and full-stack execution. The quick-start documentation indicates that the system can run PostgreSQL and the Canvas sandbox together with the API and UI, or can be deployed as the full application behind nginx.

The project documentation package also contains separate deployment, installation, user and developer documentation.

TE2 records the web application, Docker Compose configuration, developer setup/deployment documentation and architectural separation requirements as passing.

Related requirements: FR12.3–FR12.6
Final status: Fully implemented and verified – PASS

3.18 Technical Specifications

The final application is separated into frontend, backend, data-storage, Canvas-integration and AI/tool components.

Component	Final Technology / Responsibility
Frontend	React web user interface for instructors and administrators
Backend	FastAPI /api/v1 services
Database	PostgreSQL
Canvas Integration	Backend-controlled Canvas LMS interaction
AI Layer	LLM-provider integration
Tool Boundary	Optional MCP/tool-server
Deployment	Docker / Docker Compose
Full-stack serving	nginx configuration available
Backend Testing	Pytest
Frontend Testing	Vitest
End-to-End Testing	Playwright
Additional Testing	Hypothesis fuzz testing available as an opt-in suite
Version Control	GitHub

The final architecture explicitly identifies React, FastAPI, PostgreSQL, Canvas LMS, optional MCP and LLM providers as the principal system components. Automated test commands in the final repository run Pytest and Vitest coverage, Playwright end-to-end testing and optional Hypothesis fuzz testing.

3.19 Fully Dressed Use Cases

UC-01 – Authenticate User

Field	Description
Primary Actor	Instructor / Administrator
Goal	Securely authenticate and access authorized system functions.
Preconditions	A valid and enabled account exists.
Trigger	User submits login credentials.

Main Success Scenario	1. The user enters credentials. 2. The system validates them. 3. An authenticated session is established. 4. Users are granted access based on their role.
Alternative Flow	Invalid credentials are rejected. A disabled/revoked account is denied access.
Postconditions	The user enters the authorized application environment.
Related Requirements	UR1.1, UR2.1, FR1.1, FR1.10

UC-02 – Connect Canvas

Field	Description
Primary Actor	Instructor
Goal	Establish a secure connection to Canvas.
Preconditions	The instructor is authenticated.
Trigger	Instructor submits a Canvas API token.
Main Success Scenario	1. Token is submitted. 2. Backend validates the token. 3. Credentials are securely stored. 4. Canvas functionality becomes available.
Alternative Flow	Invalid token is rejected and is not used.
Postconditions	Instructors can access authorized Canvas data.
Related Requirements	UR2.2–UR2.3, FR1.4–FR1.9

UC-03 – Select and Review Course

Field	Description
Primary Actor	Instructor
Goal	Select an active Canvas course and review course information.
Preconditions	Valid Canvas connection exists.
Trigger	The instructor accesses the course interface.
Main Success Scenario	1. Courses are retrieved. 2. The instructor selects the course. 3. Selection becomes an active context. 4. Students, assignments and submission information can be retrieved.
Alternative Flow	Canvas failure results in a safe user-facing error.
Postconditions	Active course is available to dashboard and assistant workflows.
Related Requirements	UR2.4–UR2.10, FR2.1–FR2.10

UC-04 – Use Natural-Language Assistant

Field	Description
Primary Actor	Instructor

Goal	Request Canvas information or initiate a supported workflow using natural language.
Preconditions	The instructor is authenticated and relevant Canvas context is available.
Trigger	Instructor submits a natural-language request.
Main Success Scenario	1. The assistant receives a request. 2. Intent is interpreted. 3. Relevant Canvas information is retrieved. 4. Request is converted into a structured action or answer. 5. Proposed modifying actions proceed to preview.
Alternative Flow	Ambiguous requests require clarification. Unsupported requests are explained and prevented from executing.
Postconditions	Requested information is shown or a validated proposed workflow is produced.
Related Requirements	UR2.11–UR2.13, FR3.1–FR3.9

UC-05 – Create or Update Assignment

Field	Description
Primary Actor	Instructor
Goal	Create or modify a Canvas assignment.
Preconditions	The instructor has an active course context.
Trigger	Assignment creation/update is requested.
Main Success Scenario	1. Assignment information is supplied/generated. 2. Data is validated. 3. Proposed operation is previewed. 4. The instructor confirms. 5. Backend performs Canvas operation.
Alternative Flow	Instructor cancels or validation fails.
Postconditions	Canvas reflects the confirmed assignment state.
Related Requirements	UR2.14–UR2.15, FR4.1–FR4.2, FR4.4–FR4.5

UC-06 – Create Quiz

Field	Description
Primary Actor	Instructor
Goal	Create a Canvas quiz through the approved workflow.
Preconditions	The instructor is authenticated and the course selected.
Trigger	Quiz creation is requested.
Main Success Scenario	Quiz draft is created, validated, previewed and confirmed before submission to Canvas.
Alternative Flow	Invalid quiz data is rejected or the instructor cancels.
Postconditions	Confirmed quiz is created in Canvas.
Related Requirements	UR2.16, FR4.3–FR4.5

UC-07 – Draft Message or Feedback

Field	Description
Primary Actor	Instructor
Goal	Generate communication for selected students/submissions.
Preconditions	Student/submission context exists.
Trigger	Instructor requests message or feedback generation.
Main Success Scenario	1. Draft is generated. 2. Instructor reviews/edits draft. 3. Preview is shown. 4. The instructor confirms. 5. Content is posted/sent through Canvas.
Alternative Flow	Instructor cancels or edits the content before confirmation.
Postconditions	Approved communication is completed and recorded in history.
Related Requirements	UR2.18–UR2.21, FR6.1–FR6.6

UC-08 – Modify Grade

Field	Description
Primary Actor	Instructor
Goal	Modify a student grade through a controlled workflow.
Preconditions	Relevant submission exists.
Trigger	Instructor initiates grade modification.
Main Success Scenario	Proposed modification is validated, previewed, explicitly confirmed and then submitted to Canvas.
Alternative Flow	User cancels or validation rejects the action.
Postconditions	Confirmed grade modification is reflected in Canvas.
Related Requirements	UR2.22, FR5.4–FR5.5

UC-09 – Preview and Confirm Canvas Action

Field	Description
Primary Actor	Instructor
Goal	Understand and explicitly authorize a Canvas-modifying operation.
Preconditions	A validated Canvas-modifying action exists.
Trigger	The system prepares action for execution.
Main Success Scenario	1. Preview describes action. 2. Affected Canvas resources are shown. 3. Reversibility/risk is communicated. 4. The instructor confirms. 5. Backend executes operation.

Alternative Flow	Instructor cancels; no Canvas write occurs.
Postconditions	Action either executes with explicit approval or remains unexecuted.
Related Requirements	UR2.23–UR2.25, FR7.1–FR7.6

UC-10 – Review History or Reverse Supported Action

Field	Description
Primary Actor	Instructor
Goal	Review previous actions and reverse an action where supported.
Preconditions	Previous workflow records exist.
Trigger	Instructor opens history.
Main Success Scenario	The system displays request, action, status, time and affected resources. Reversal information is displayed and supported actions may be reversed through the approved workflow.
Alternative Flow	The system explains that reversal is unavailable.
Postconditions	The user understands the historical state; supported reversal may be performed.
Related Requirements	UR2.26–UR2.28, FR8.1–FR8.5

UC-11 – Administer System

Field	Description
Primary Actor	Administrator
Goal	Maintain accounts and review operational/system information.
Preconditions	Administrator is authenticated.
Trigger	Administrator accesses administrative functionality.
Main Success Scenario	Administrator manages instructor accounts, reviews audit/health information and investigates failed operations.
Alternative Flow	Unauthorized users are denied administrative functionality.
Postconditions	Administrative state is updated or system information is reviewed.
Related Requirements	UR1.2–UR1.7, FR12.1–FR12.2

3.20 Requirements and Feature Traceability

Feature Area	User Requirements	Functional Requirements	Contributors identified in TE2	Final Status
Authentication & Account Management	UR1.1–UR1.3, UR2.1	FR1.1–FR1.3, FR1.10	Jessica, Mridul, Bao, Yerdana	PASS
Canvas Credential Management	UR2.2–UR2.3	FR1.4–FR1.9	Jessica, Mridul, Bao	PASS

Course Dashboard & Data Access	UR2.4–UR2.10, UR2.17	FR2.1–FR2.10	Jessica, Mridul, Rhythm, Bao	PASS
Natural-Language Assistant	UR2.11–UR2.13	FR3.1–FR3.9	Jessica, Mridul, Yerdana	PASS
Assignment Creation/Update	UR2.14–UR2.15	FR4.1–FR4.2, FR4.4–FR4.5	Jessica, Mridul, Yerdana	PASS
Quiz Creation	UR2.16	FR4.3–FR4.5	Jessica, Mridul, Yerdana	PASS
Submission Analysis	UR2.7–UR2.8, UR2.17	FR5.1–FR5.3	Mridul, Rhythm, Bao	PASS
Grade Modification	UR2.22	FR5.4–FR5.5	Jessica, Mridul	PASS
Canvas Messaging	UR2.18–UR2.19	FR6.1–FR6.3, FR6.6	Jessica, Mridul, Yerdana	PASS
Feedback	UR2.20–UR2.21	FR6.4–FR6.6	Jessica, Mridul, Yerdana	PASS
Dry-Run / Confirmation	UR2.23–UR2.25	FR7.1–FR7.6	Jessica, Mridul, Rhythm, Yerdana	PASS
History / Reversal	UR2.26–UR2.28	FR8.1–FR8.5	Jessica, Mridul, Rhythm, Bao, Yerdana	PASS
AI / MCP Validation	—	FR9.1–FR9.6	Jessica, Mridul, Rhythm	PASS
Security / Privacy	—	FR10.1–FR10.5	Jessica, Rhythm, Bao	PASS
Reliability / Errors	UR2.29	FR11.1–FR11.4	Jessica, Rhythm	PASS
Admin Health / Deployment	UR1.4–UR1.7	FR12.1–FR12.6	Jessica, Bao, Yerdana, Rhythm	PASS

The TE2 report records every listed user and functional requirement above as passing.

3.21 Section Summary

The final Canvas Helper AI Assistant delivers the complete set of course-management, AI-assisted, administrative and safety workflows described in the project's requirements and verified during Testing Event 2. The system supports both information-retrieval and Canvas-modifying workflows while maintaining a strict separation between AI reasoning and direct Canvas execution.

The combination of backend validation, explicit user previews, confirm-before-write controls, encrypted credential handling, action history, error management and restricted tool execution ensures that AI assists the instructor without removing human responsibility for consequential actions. This principle is reflected both in the final architecture and in the project's explicit scope constraint that autonomous Canvas modifications and direct validation-bypassing Canvas access are not supported.

4. Status of the System Features

4.1 Overview

The final Canvas Helper AI Assistant was evaluated against the complete set of user and functional requirements defined for the project. The purpose of this section is to connect the feature descriptions presented in Section 3 with evidence showing whether each requirement was successfully implemented and tested.

The final Testing Event 2 report records every listed user requirement and functional requirement as **PASS**. Depending on the requirement, verification included unit testing, integration testing, system functionality testing, operational acceptance testing, usability testing, and acceptance testing.

The final repository also defines three major automated testing workflows: make test for backend Pytest and frontend Vitest coverage, make test-e2e for Playwright end-to-end testing, and make test-fuzz for optional Hypothesis fuzz testing.

Together, the automated tests, manual testing activities, and Testing Event 2 requirement verification were used to validate the final system.

4.2 Testing Strategy

The system was validated using multiple testing approaches rather than relying on one testing method. This was important because the application combines frontend interaction, backend APIs, database operations, Canvas integration, AI-assisted workflows, and safety-sensitive confirmation logic.

The Testing Event 2 report identifies the following testing categories:

Code	Testing Type	Purpose
UN	Unit Testing	Validates individual functions, services, and components independently.
I	Integration Testing	Confirms that system components work correctly together.
S	System Functionality Testing	Tests complete features at system level.
O	Operational Acceptance Testing	Evaluates operational and administrative behaviour.
US	Usability Testing	Evaluates the system from the user-interface and workflow perspective.
A	Acceptance Testing	Confirms that implemented behaviour satisfies project requirements.

These testing categories are explicitly defined in the TE2 report.

The automated test infrastructure complements these testing activities. The final repository specifies Pytest and Vitest coverage for normal automated testing, Playwright for end-to-end workflows, and Hypothesis for optional fuzz testing.

4.3 User Requirement Status

4.3.1 Administrator Requirements

All administrator-facing user requirements were verified as passing.

Requirement	Description	Result
UR1.1	Secure administrator login and access to administrative functions	PASS
UR1.2	Create, update, enable, disable and manage instructor accounts	PASS
UR1.3	Revoke instructor access	PASS
UR1.4	Monitor system health and operational status	PASS
UR1.5	Review audit logs and security-related events	PASS
UR1.6	Review failed Canvas and AI operations	PASS
UR1.7	Troubleshoot without accessing sensitive credentials	PASS

TE2 records all seven requirements as passing.

The administrator feature set therefore provides account administration in addition to operational monitoring and troubleshooting. These requirements correspond to the administrator functions described in Section 3.

4.3.2 Instructor Requirements

All instructor-facing requirements were also recorded as passing in Testing Event 2.

Requirement	Description	Result
UR2.1	Create account, authenticate and access application	PASS
UR2.2	Securely connect Canvas API token	PASS
UR2.3	Replace or remove Canvas API token	PASS
UR2.4	View connected Canvas courses	PASS
UR2.5	Select an active course	PASS
UR2.6	View course information, students, assignments and submissions	PASS
UR2.7	Analyze missing submissions	PASS
UR2.8	Analyze late submissions	PASS
UR2.9	Search and filter course information	PASS
UR2.10	Access course activity dashboard	PASS
UR2.11	Use natural-language requests	PASS
UR2.12	Ask questions about course data	PASS
UR2.13	Receive explanation of proposed actions	PASS
UR2.14	Create Canvas assignments	PASS
UR2.15	Update assignment details and dates	PASS
UR2.16	Create Canvas quizzes	PASS
UR2.17	List assignment submissions	PASS
UR2.18	Draft Canvas messages	PASS
UR2.19	Review and edit messages	PASS
UR2.20	Draft feedback comments	PASS
UR2.21	Review and approve feedback	PASS
UR2.22	Modify grades through approved workflow	PASS
UR2.23	Preview Canvas-modifying actions	PASS
UR2.24	Explicitly approve modifying actions	PASS
UR2.25	Cancel proposed actions	PASS

UR2.26	View history of previous actions	PASS
UR2.27	View previous assistant requests and outcomes	PASS
UR2.28	Determine whether an action can be reversed	PASS
UR2.29	Receive safe and clear failure feedback	PASS

Requirements UR2.1–UR2.22 are recorded as passing on the first page of the TE2 report. Requirements UR2.23–UR2.29 are likewise recorded as passing.

Therefore, the final system provides complete coverage of the instructor user requirements defined for the project.

4.4 Functional Requirement Status

4.4.1 Authentication, Account Management and Canvas Token Setup

The authentication and Canvas credential subsystem passed all requirements from FR1.1 through FR1.10.

The validated behaviours include authenticated instructor access, administrator account management, access revocation, Canvas token entry/removal, encryption, validation, prevention of token exposure, correct use of each instructor's token, user isolation and logout/session expiration.

Status: Fully implemented and verified – PASS

4.4.2 Course Dashboard and Canvas Data Access

All requirements in this feature group passed testing. The system successfully retrieves courses, students, assignments and submissions; maintains the active course; provides dashboard summaries; identifies submission states; offers workflow shortcuts; and treats Canvas as the authoritative source for course and assignment information.

Status: Fully implemented and verified – PASS

4.4.3 Natural-Language Assistant

The assistant successfully passed testing for its chat-style interaction model, Canvas workflow support, structured action interpretation, clarification behaviour, interpreted-intent display, supported workflows, unsupported-request handling, prevention of fabricated Canvas information, and active-course scoping.

The supported workflow list verified by TE2 includes listing courses, students, assignments and submissions; assignment creation; assignment date updates; student messaging; feedback generation; quiz creation; and grade modification.

Status: Fully implemented and verified – PASS

4.4.4 Assignment and Quiz Management

Assignment and quiz workflows passed all five functional requirements.

The validated system supports assignment creation, assignment modification, quiz creation, validation of assignment/quiz information and preview/confirmation before Canvas execution.

Status: Fully implemented and verified – PASS

4.4.5 Submission Analysis and Grading

All five requirements associated with submission analysis and grading passed.

The final system identifies missing and late submissions, displays submission summaries and affected students, supports approved grade modifications, and requires preview and confirmation for grade changes.

Status: Fully implemented and verified – PASS

4.4.6 Messaging and Feedback

Canvas messaging and feedback workflows passed all six requirements.

Instructors can generate draft messages, review and edit them, preview and confirm message delivery, generate feedback comments, confirm feedback posting and retain messaging/feedback activity in action history.

Status: Fully implemented and verified – PASS

4.4.7 Dry-Run Preview and Confirmation

All Canvas-modifying operations are subject to the system's dry-run confirmation model.

TE2 verified that modifying actions generate previews, identify affected resources, communicate reversibility, require explicit confirmation, support cancellation and classify actions according to risk.

This directly validates the project's architectural **confirm-before-write** principle. The final architecture identifies mandatory human confirmation as one of the system's primary design decisions.

Status: Fully implemented and verified – PASS

4.4.8 Action History and Reversal

The final system successfully supports user-visible action history and limited reversal.

TE2 verifies that history contains original requests, actions, execution information, timestamps and affected resources. Assistant requests are linked with their resulting actions, and supported actions can be reversed where safe and practical. Where reversal is unavailable, the system communicates that limitation.

Status: Fully implemented and verified – PASS

4.4.9 AI Validation and MCP Integration

The AI safety boundary also passed all tested requirements.

The model cannot directly invoke unrestricted Canvas operations. Canvas operations are instead controlled by the backend/tool layer, AI-generated actions undergo validation, unsupported or unsafe operations are rejected, and only approved operations are exposed through the tool boundary.

The final architecture describes MCP as an optional tool and credential boundary rather than the owner of the application's business logic.

Status: Fully implemented and verified – PASS

4.4.10 Security and Privacy

All security/privacy requirements passed TE2 testing.

The tested requirements include encrypted Canvas and AI credentials, prevention of credential leakage through logs or user-facing responses, authorization of student information, minimization of student-identifiable information sent to AI services, and backend validation of all Canvas-changing operations.

Status: Fully implemented and verified – PASS

4.4.11 Reliability and Error Handling

All reliability requirements passed.

The application provides meaningful errors, safely handles Canvas API failures, handles incomplete or unavailable Canvas information, and protects against duplicate execution of Canvas-modifying operations.

Status: Fully implemented and verified – PASS

4.4.12 Administration and Deployment

The system's deployment and operational requirements also passed.

TE2 verified administrator health information, failed Canvas/AI operation monitoring, web-based delivery, Docker Compose deployment, developer/deployment documentation and separation of the application's architectural components.

The root project README provides runnable development and deployment workflows using Make and Docker Compose, while the final documentation package includes separate deployment and developer documentation.

Status: Fully implemented and verified – PASS

4.5 Scope Constraint Verification

The project also defines explicit constraints describing behaviours that the application must **not** support.

These constraints are important because successful implementation does not mean maximizing automation. In this system, preventing unsafe functionality is itself a requirement.

Constraint	Result
No autonomous Canvas modifications without instructor approval	PASS
No Canvas execution that bypasses backend validation	PASS
No institution-wide Canvas administration	PASS
No unrestricted deletion of Canvas resources	PASS
No arbitrary user-defined Canvas API calls	PASS

TE2 records the scope constraints as passing.

These results demonstrate that the final implementation preserves the intended safety boundary rather than exposing unrestricted Canvas functionality.

4.6 Overall Feature Status Matrix

System Feature	Implementation Status	TE2 Result
Authentication	Complete	PASS
Instructor account management	Complete	PASS
Canvas connection/token management	Complete	PASS
Course retrieval and active course	Complete	PASS
Dashboard	Complete	PASS
Student/assignment/submission retrieval	Complete	PASS
Missing and late submission analysis	Complete	PASS
Natural-language assistant	Complete	PASS
Assignment creation	Complete	PASS
Assignment update	Complete	PASS
Quiz creation	Complete	PASS
Canvas messaging	Complete	PASS
Feedback generation/posting	Complete	PASS
Grade modification	Complete	PASS
Dry-run preview	Complete	PASS
Explicit confirmation/cancellation	Complete	PASS
Action history	Complete	PASS
Limited reversal	Complete	PASS
Administrative audit and health monitoring	Complete	PASS
Failed-operation monitoring	Complete	PASS
AI validation	Complete	PASS
MCP/tool execution boundary	Complete	PASS
Security and privacy controls	Complete	PASS
Error handling/reliability	Complete	PASS
Docker-based deployment	Complete	PASS
Documentation/setup support	Complete	PASS

The TE2 evidence supports the conclusion that each requirement represented by these feature areas passed its assigned validation activities.

4.7 Automated Test Environment

Automated testing is integrated into the final repository and can be run through the project's Makefile.

The final repository documents the following commands:

make test
make test-e2e
make test-fuzz

make test runs backend Pytest tests and frontend Vitest coverage. make test-e2e executes Playwright end-to-end testing, while make test-fuzz executes the optional Hypothesis fuzz-test suite.

This layered strategy provides complementary verification:

- unit/component tests verify isolated logic;
- integration tests verify communication between components;
- end-to-end tests exercise complete user workflows;
- fuzz testing checks behaviour against a broader range of generated inputs; and
- Testing Events validate usability and requirement acceptance using the running application.

4.8 Testing Event 2 Results

Testing Event 2 served as the final documented requirement-level verification artifact provided for this report.

The report maps every UR and FR to applicable testing types, a PASS/FAIL result and the contributors involved. Across the report, all listed requirements are marked **PASS**.

This provides direct evidence that the final feature implementation satisfies the project's documented requirement set.

		REQUIREMENTS	TYPE OF TESTING(I = Integration Testing S = System Functionality Testing O = Operational Acceptance Testing UN = Unit Testing US = Usability Testing A = Acceptance Testing,- = not applied)	PASS OR FAIL	CONTRIBUTOR	STATUS
2		***USER REQUIREMENTS***				
3		***INSTRUCTOR***				
4	1.1	Must log in securely and access administrative functions	UN,I,S,US,A	PASS	JESSICA,MRIDUL,BAO	☑
5	1.2	Must be able to create, update, enable, disable, and manage instructor accounts	UN,I,S,US,A	PASS	JESSICA,MRIDUL,BAO	☑
6	1.3	Must be able to revoke instructor access when required.	UN,I,S,US,A	PASS	JESSICA,MRIDUL,BAO	☑
7	1.4	Must be able to monitor system health and operational status.	UN, I, S, O	PASS	BAO	☑
8	1.5	Must be able to review audit logs and security-related events.	UN, I, S, O	PASS	BAO	☑
9	1.6	Must be able to review failed Canvas and AI operations.	UN, I, S, O	PASS	RHYTHM	☑
10	1.7	Must be able to troubleshoot the system without accessing sensitive credentials.	UN, I, S, O	PASS	BAO	☑
11		***ADMINISTRATOR***				
12	2.1	Must be able to securely create account, authenticate and access the application	UN,I,S,US,A	PASS	JESSICA,MRIDUL,YERDANA	☑
13	2.2	Must be able to securely connect a Canvas API token.	UN,I,S,US,A	PASS	JESSICA,MRIDUL,BAO	☑
14	2.3	Must be able to replace or remove a stored Canvas API token.	UN,I,S,US,A	PASS	JESSICA,MRIDUL,BAO	☑
15	2.4	Must be able to view Canvas courses associated with the connected token.	UN,I,S,US,A	PASS	RHYTHM,MRIDUL,BAO	☑
16	2.5	Must be able to select an active course.	UN,I,S,US,A	PASS	RHYTHM,MRIDUL,BAO	☑
17	2.6	Must be able to view course information, students, assignments, and submissions.	UN,I,S,US,A	PASS	JESSICA,MRIDUL,BAO,RHYTHM	☑
18	2.7	Must be able to analyze missing submissions.	UN,I,S,US,A	PASS	RHYTHM,MRIDUL,BAO	☑
19	2.8	Must be able to analyze late submissions.	UN,I,S,US,A	PASS	RHYTHM,MRIDUL,BAO	☑
20	2.9	Must be able to search and filter course information	UN,I,S,US,A	PASS	RHYTHM,MRIDUL,BAO	☑
21	2.10	Must be able to access a dashboard summarizing course activity.	UN,I,S,US,A	PASS	RHYTHM,MRIDUL,BAO	☑
22	2.11	Must be able to interact with the system using natural-language requests.	UN,I,S,US,A	PASS	JESSICA,MRIDUL,YERDANA	☑
23	2.12	Must be able to ask questions about course data.	UN,I,S,US,A	PASS	JESSICA,MRIDUL,YERDANA	☑
24	2.13	Must be able to receive explanations of proposed actions before execution.	UN,I,S,US,A	PASS	JESSICA,MRIDUL,YERDANA	☑
25	2.14	Must be able to create Canvas assignments.	UN,I,S,US,A	PASS	JESSICA,MRIDUL,YERDANA	☑
26	2.15	Must be able to update assignment details and dates.	UN,I,S,US,A	PASS	JESSICA,MRIDUL,YERDANA	☑
27	2.16	Must be able to create Canvas quizzes.	UN,I,S,US,A	PASS	JESSICA	☑
28	2.17	Must be able to list submissions for assignments	UN,I,S,US,A	PASS	RHYTHM,MRIDUL,BAO	☑
29	2.18	Must be able to draft Canvas messages to selected students.	UN,I,S,US,A	PASS	JESSICA,MRIDUL,YERDANA	☑
30	2.19	Must be able to review and edit messages before sending.	UN,I,S,US,A	PASS	JESSICA,MRIDUL,YERDANA	☑
31	2.20	Must be able to draft feedback comments for selected submissions	UN,I,S,US,A	PASS	JESSICA	☑
32	2.21	Must be able to review and approve feedback before posting.	UN,I,S,US,A	PASS	JESSICA	☑
33	2.22	Must be able to modify grades through approved workflows.	UN,I,S,US,A	PASS	MRIDUL	☑
34	2.23	Must be able to preview Canvas-modifying actions before execution.	UN,I,S,US,A	PASS	JESSICA,MRIDUL,YERDANA	☑
35	2.24	Must explicitly approve Canvas-modifying actions before execution.	UN,I,S,US,A	PASS	RHYTHM,MRIDUL,YERDANA	☑
36	2.25	Must be able to cancel a proposed action before execution.	UN,I,S,US,A	PASS	RHYTHM,MRIDUL,YERDANA	☑
37	2.26	Must be able to view a history of previous actions.	UN,I,S,US,A	PASS	RHYTHM,MRIDUL,YERDANA	☑
38	2.27	Must be able to view previous assistant requests and their outcomes.	UN,I,S,US,A	PASS	RHYTHM,MRIDUL,YERDANA	☑
39	2.28	Must be able to determine whether a previous action can be reversed	UN,I,S,US,A	PASS	MRIDUL	☑
40	2.29	Must receive clear feedback when requests cannot be completed safely.	UN,I,S,US,A	PASS	RHYTHM,MRIDUL,YERDANA	☑
41		***FUNCTIONAL REQUIREMENTS***				
42		***Authentication, Account Management, and Canvas API Token Setup***				
43	1.1	The system must support instructor login before any Canvas-related functionality is accessible.	UN,I,S,US,A	PASS	JESSICA,MRIDUL,BAO	☑

Figure 4.1 – Testing Event 2 Requirements/Testing Matrix

4.9 User Testing Relationship

The project repository contains separate Testing Event 1 findings, UX notes and Testing Event 2 testing artifacts. The final testing documentation explicitly references these artifacts as part of the project's final testing package.

Detailed analysis of usability findings, how the team responded to those findings, and the results of re-validation should be presented in **Section 5 – UX Analysis**, rather than duplicated in full here.

For Section 4, the important conclusion is that usability and acceptance testing were incorporated into requirement validation, with the TE2 matrix listing **US** and **A** testing for most end-user functionality.

4.10 Known Bugs and Outstanding Issues

The submitted TE2 requirement matrix contains **no failed requirements**. Every listed requirement and scope constraint is marked PASS.

At the time of final requirement verification, no requirements were recorded as failed in the Testing Event 2 report. All documented user, functional, and scope requirements achieved a PASS result.

Outstanding risks: None identified at final freeze beyond ordinary maintenance items such as dependency updates and continued Canvas API compatibility monitoring.

Known bugs at final submission: None currently known. Any non-blocking defects discovered after submission should be tracked as GitHub issues and addressed in follow-up maintenance.

4.11 Section Summary

The final testing results demonstrate that the Canvas Helper AI Assistant successfully implemented and validated the full documented project scope. All user requirements, functional requirements and safety-related scope constraints represented in Testing Event 2 achieved PASS results.

Testing was conducted at multiple levels, including unit, integration, system functionality, usability, acceptance and operational acceptance testing, supplemented by automated Pytest, Vitest, Playwright and optional Hypothesis test suites.

The results indicate that the final system meets the functional scope described in Section 3 while maintaining the project's central safety properties, particularly backend validation and explicit human confirmation before Canvas-modifying actions.

5. UX Analysis

5.1 Overview

User experience evaluation was conducted throughout development to ensure that the Canvas Helper AI Assistant was not only technically functional but also understandable, efficient, and safe for instructors and administrators to use. The team used feedback from demonstrations, client feedback, Testing Event 1, subsequent implementation work, and Testing Event 2 to identify usability problems and make concrete improvements to the final system.

The final repository preserves both the detailed Testing Event 1 findings report and a separate UX report, while Testing Event 2 was used for later requirement-level validation. The final testing documentation also explicitly references TE1 findings, UX notes, and the TE2 testing report as the project's primary testing artifacts.

The UX process therefore followed an iterative cycle:

Initial Design → User Testing → Identify Pain Points → Implement Changes → Re-test / Validate

This approach allowed feedback to influence the actual implementation rather than being collected only for documentation purposes.

5.2 User Testing Sessions

The team conducted formal usability evaluation during the project's testing events in addition to receiving feedback from the client and course presentation sessions.

Number of testers: 12

Tester profile: Users participating in the COSC 499 testing event who evaluated the Canvas Helper AI Assistant from the perspective of its intended instructor workflows.

The testing focused primarily on whether users could understand and complete the application's core workflows, including authentication, navigation, Canvas data access, AI-assisted actions, previews, confirmations, and administrative interactions.

Testing Event 2 later evaluated the completed user requirements using unit, integration, system, usability, and acceptance testing where applicable. All instructor-facing user requirements were recorded as passing.

5.3 Key Scenarios and Tasks Tested

The usability evaluation covered the workflows that instructors and administrators would most frequently encounter when using the system.

Scenario	Purpose of Testing
Login and account access	Determine whether authentication and error messages were understandable
Dashboard navigation	Evaluate whether important functionality could be found easily
Course selection	Determine whether users understood the currently selected Canvas course
Viewing Canvas data	Test presentation of courses, students, assignments and submissions

Natural-language assistant	Determine whether users understood how to request actions using conversational input
Assignment creation/update	Evaluate structured editing and modification workflows
Student messaging	Test generation, editing and review of messages
Dry-run preview	Determine whether proposed actions were understandable before execution
Confirmation/cancellation	Evaluate whether users understood consequences before modifying Canvas
Admin account management	Test creation, disabling and management of instructor accounts
Error handling	Determine whether errors provided useful information and recovery guidance

These workflows correspond closely to the final user requirements validated during TE2, including course access, natural-language interaction, assignment management, messaging, previews, confirmations, history and error feedback.

5.4 Major UX Findings and Team

Responses

5.4.1 Finding 1 – Confirmation Previews Contained Too Much Technical Information

One of the most important pieces of feedback concerned the confirmation and preview workflow. Although users appreciated being able to review actions before execution, the preview screens presented more technical information than was necessary for making a decision.

This created a usability problem because the user primarily needed to understand:

- what was going to happen;
- which Canvas resource would be affected;
- what information would change; and
- whether the action could be reversed.

Technical details could distract from these key decisions.

Team Response

The team revised the confirmation experience so that essential user-facing information received greater visual emphasis while lower-level technical information could be separated or placed in a less prominent area.

The final confirmation design also uses information about the action's risk and reversibility rather than treating every operation identically.

This improvement directly aligns with the final requirement that previews summarize the proposed action and affected Canvas resources and communicate whether the action can be reversed.

TE2 subsequently recorded all dry-run preview and confirmation requirements as passing.

5.4.2 Finding 2 – Confirmation Should Reflect the Risk of the Action

Feedback also indicated that the same level of interruption was not appropriate for every operation.

For example, simply reading course information carries significantly less risk than changing an assignment, sending a message, posting feedback, or changing a grade. Requiring the same confirmation experience for both could make normal navigation unnecessarily cumbersome while also failing to emphasize genuinely high-risk operations.

Team Response

The team incorporated **risk-sensitive confirmation behaviour**.

Actions were classified according to their impact, including:

- read-only actions;
- modifying actions; and
- irreversible or higher-risk actions.

Higher-impact operations require stronger confirmation, while read-only interactions can proceed with less interruption.

This behaviour became an explicit final functional requirement: FR7.6 requires the system to classify actions as read-only, modifying, or irreversible.

The final architecture additionally identifies mandatory human confirmation before Canvas writes as one of the system's core design decisions.

5.4.3 Finding 3 – Users Needed Editable Structured Forms

Early AI-assisted workflows relied heavily on generated content. User feedback demonstrated that instructors needed a simple way to modify AI output rather than accepting generated content exactly as produced.

This was especially relevant for:

- assignment creation;
- assignment updates;
- quizzes;
- student messages; and
- feedback.

AI can accelerate content creation, but instructors still need precise control over dates, descriptions, wording, and other structured information.

Team Response

The team strengthened structured editing workflows so that generated content could be reviewed and modified before execution.

For assignments and quizzes, information is validated before submission and must proceed through preview and confirmation.

Similarly, the messaging requirements explicitly support reviewing and editing messages before sending.

By the final Testing Event 2, assignment creation, assignment updates, quiz creation, validation and confirmation all achieved PASS results. Messaging and feedback workflows also passed.

5.4.4 Finding 4 – Selected Course Context Needed to Persist

Another usability issue concerned course context.

Because instructors can have access to multiple Canvas courses, losing track of which course is currently active creates a serious usability and safety problem. A user may understand an AI request differently depending on the active course.

For example:

"Show me students with missing assignments"

is ambiguous unless the system knows which course the instructor is currently working with.

Team Response

The final system maintains an **active course** and uses that course as the default context for subsequent workflows.

The original requirement states that instructors must be able to select an active course, and the Natural-Language Assistant must scope requests to the selected course unless another course is explicitly specified.

The final repository's traceability documentation also specifically identifies the active-course preference as an implemented feature.

TE2 confirmed that both course selection and active-course behaviour passed testing.

5.4.5 Finding 5 – AI and Canvas Errors Needed to Be More Understandable

User feedback indicated that technical or generic error responses were not sufficient when an AI or Canvas operation failed.

A useful error message should explain what happened and, where possible, tell the instructor what they can do next.

This issue was particularly important because the application communicates with several independent components, including Canvas, the backend, the database, and AI providers.

Team Response

The team improved user-facing error handling and added safeguards for unavailable or incomplete Canvas information.

The final reliability requirements specifically require:

- meaningful user-facing error messages;
- safe handling of Canvas API failures;
- safe handling of incomplete or unavailable Canvas data; and
- prevention of duplicate modifying actions.

The instructor requirements also specify that users must receive clear feedback when a request cannot safely be completed.

All corresponding reliability and error-handling requirements passed TE2.

5.5 Positive UX Findings

User testing did not only identify problems. Several aspects of the interface were positively received.

The dashboard-based design and organized navigation helped users locate major workflows from a central interface. Users also responded positively to the system's clear structure and the ability to preview AI-generated actions rather than allowing the AI to silently modify Canvas.

The disabled-account workflow was another useful example of clear system feedback: users were explicitly informed when access had been disabled instead of receiving a generic login failure.

The free-form natural-language interaction was also identified as a promising component because it allowed instructors to initiate tasks using conversational requests rather than navigating every workflow manually.

These strengths were retained while the team focused improvements on confirmation design, structured editing, course context, and error messaging.

5.6 How Feedback Changed the Final Product

The value of the UX process was not simply that feedback was collected. The major feedback items resulted in concrete changes to the final system.

User Feedback / Observation	Change Made	Final Validation
Confirmation screens were too technical	Simplified primary confirmation information	Preview requirements PASS
Different actions have different risk levels	Added risk-sensitive confirmation and reversibility information	FR7.1–FR7.6 PASS
AI output needed to remain editable	Added/strengthened structured forms and editable drafts	Assignment/message workflows PASS
Course selection could be lost between workflows	Preserved selected active course context	Course-selection requirements PASS
AI/Canvas failures were difficult to understand	Improved user-facing error handling	FR11.1–FR11.4 PASS
Users needed control before AI actions	Maintained explicit confirm-before-write architecture	Scope and confirmation requirements PASS

Testing Event 2 confirms successful validation of course selection, natural-language interaction, assignment workflows, messaging, confirmation and error-handling requirements after development and refinement.

5.7 Re-testing and Re-validation

Following the initial usability feedback, the team continued development and incorporated UX changes before later validation.

Testing Event 2 acted as an important re-validation stage because it evaluated the final implementation against the project's complete requirement set.

The features most directly associated with earlier UX concerns successfully passed testing:

Re-tested Area	TE2 Result
Active course selection	PASS
Dashboard/course information	PASS
Natural-language assistant	PASS
Explanation of proposed actions	PASS
Assignment creation/update	PASS
Editable Canvas messages	PASS
Dry-run preview	PASS
Explicit confirmation	PASS
Cancel action	PASS
Clear unsafe-request feedback	PASS

These results are recorded in the TE2 user-requirement matrix.

The results provide evidence that UX findings were not simply documented but were addressed in the implementation and later verified.

5.8 UX Reflection

What Surprised the Team?

One important observation was that providing **more information does not necessarily create greater user trust**.

The team initially treated detailed confirmation information as beneficial because it increased transparency. User feedback showed that excessive implementation details could actually make an important decision harder to understand.

The lesson was that transparency must be presented at the appropriate level. Users need enough information to understand the consequence of an action, but implementation-level detail should not compete with that information.

What Feedback Contradicted Initial Assumptions?

One assumption was that a uniform confirmation workflow would provide the safest and most consistent experience.

Feedback demonstrated that consistency does not necessarily mean treating every action identically. Users expect the amount of interruption to correspond to the consequences of the action.

A read-only course query does not require the same warning as a grade change or Canvas modification.

This led to the stronger distinction between read-only, modifying, and potentially irreversible actions now represented in the system's confirmation requirements.

What Would the Team Do Differently?

In future projects, UX testing should begin earlier and focus on complete user workflows rather than primarily evaluating isolated pages or features.

For an AI-assisted application in particular, testing should investigate not only whether users can perform an action but also:

- whether they understand what the AI is proposing;
- whether they understand the consequences;
- whether they know when AI output can be edited;
- whether confirmation provides the correct amount of information;
- whether errors explain the next step; and
- whether users remain aware of the current system context.

The project demonstrated that AI UX is closely connected to trust, safety, and user control. As a result, future iterations would involve users earlier when designing confirmation flows, AI explanations, editing interactions, and error recovery.

5.9 UX Analysis Conclusion

UX evaluation significantly influenced the final Canvas Helper AI Assistant.

User testing identified issues surrounding excessive technical detail, confirmation behaviour, editability of AI output, persistence of course context, and error messaging. Instead of leaving these observations as recommendations, the team incorporated them into later design and implementation work.

The final system therefore provides editable AI-assisted workflows, persistent course context, clearer error handling, risk-aware action previews, cancellation controls, and explicit confirmation before Canvas-modifying operations.

Testing Event 2 subsequently recorded the corresponding requirements as passing, providing evidence that these improvements were incorporated successfully into the final implementation.

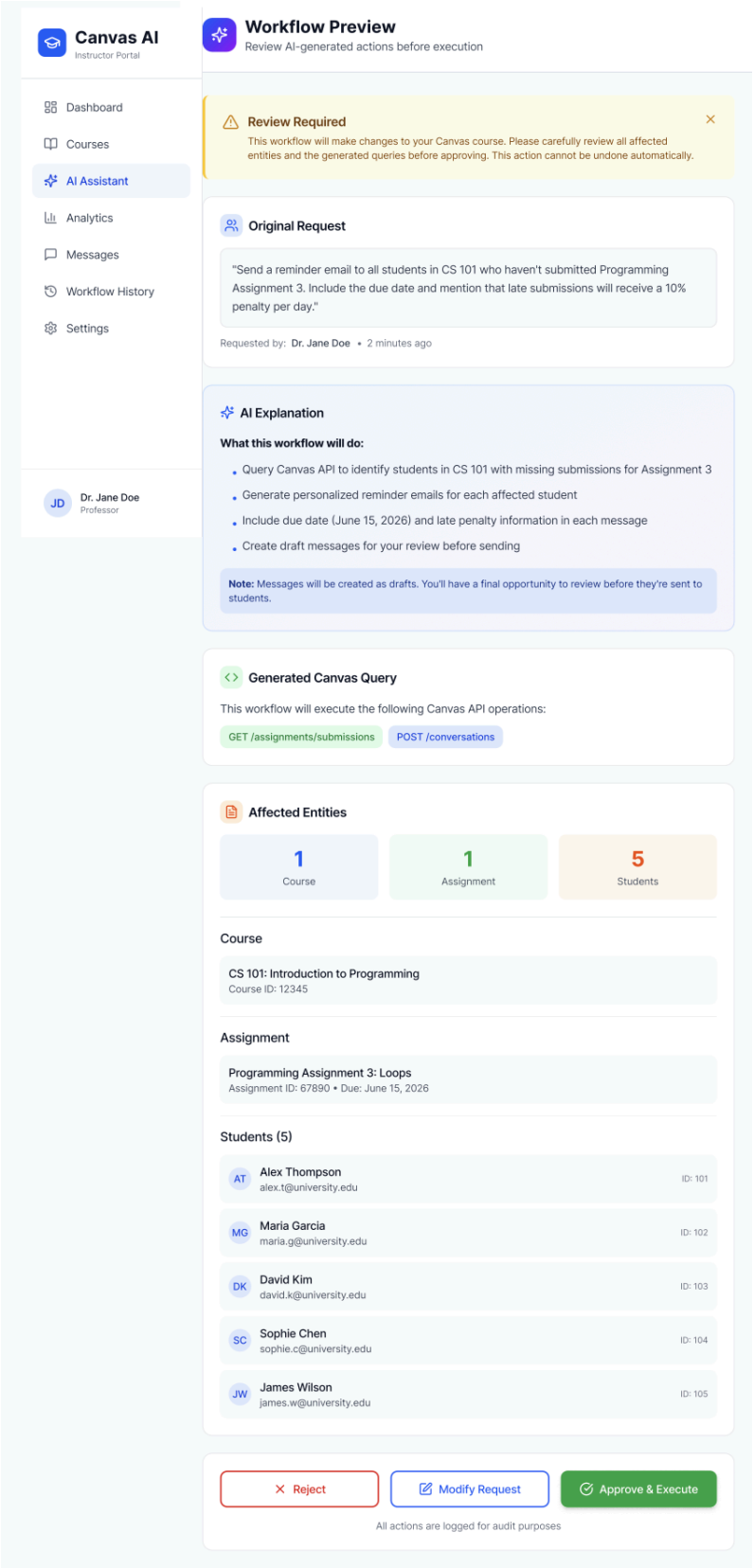


Figure 5.1: The initial preview design.

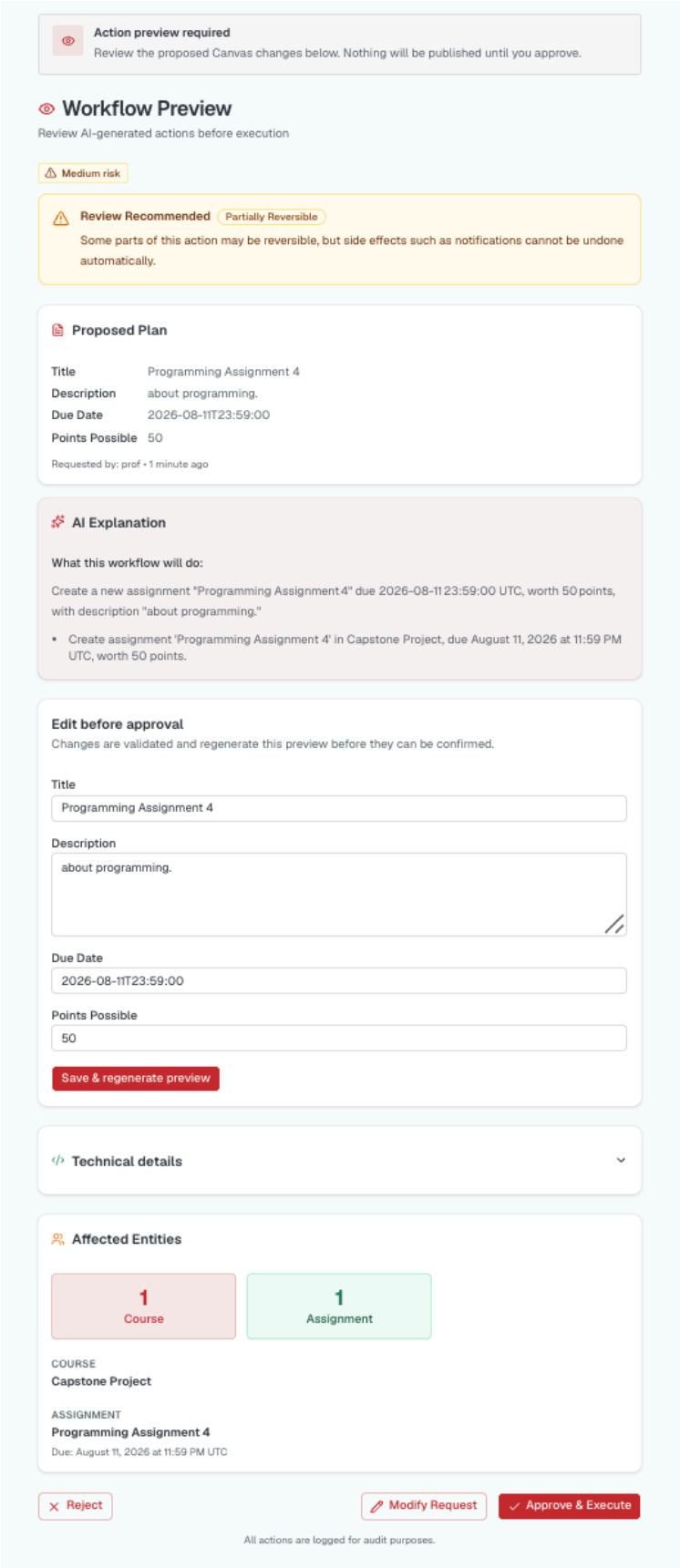


Figure 5.2: The final, risk-aware confirmation design, featuring collapsed technical details and an editable structured form, directly addressing our TE1 usability findings.

6. Project Management Activities and WBS/FBS

6.1 Project Management Approach

The Canvas Helper AI Assistant was managed as a multi-component software engineering project involving frontend development, backend development, AI/LLM integration, Canvas API integration, testing, deployment, documentation, and user experience refinement. The project plan assigned primary responsibilities across the five team members, with Mridul focusing on backend, DevOps and architecture; Jessica on frontend, integration and backend development; Bao on backend development and documentation; Yerdana on AI/LLM integration and validation; and Rhythm on frontend development, QA and accessibility.

The team used an iterative Agile-style workflow supported by GitHub Projects, pull requests, milestone tracking, weekly coordination meetings, testing cycles, and individual/team work logs. The project plan explicitly identified GitHub Projects Kanban, weekly sync meetings, pull-request reviews and milestone tracking as the primary mechanisms for coordinating work across the five-person team.

Project activities were divided into manageable features and work packages so that individual members could work in parallel while still integrating their work into common system workflows. This was particularly important because the system contained several dependent layers: frontend interfaces depended on backend APIs, AI workflows depended on validation and Canvas services, and modifying Canvas actions depended on the preview and confirmation mechanisms.

6.2 Planning and Milestone Management

The team maintained a milestone-oriented development plan from initial planning through final delivery. The project plan defined major checkpoints for project planning, architecture/design, prototype development, the first MVP feature set, integration/testing, full system testing and final submission.

As implementation progressed, work was further divided into development cycles and individual GitHub issues. This allowed the team to respond to scope changes, client feedback and testing results without treating the original plan as fixed.

The project-management process generally followed the cycle:

Plan work → Assign issues → Implement → Open pull request → Review/integrate → Test → Record actual effort → Reprioritize next cycle

This approach supported continuous delivery of functionality rather than postponing integration until the end of the project.

6.3 Work Breakdown Structure (WBS)

The Work Breakdown Structure organizes the project according to the work required to deliver the final system.

Level 1 – Canvas Helper AI Assistant

Level 2 – Major Work Packages

WBS ID	Work Package	Major Activities
1.0	Project Planning and Requirements	Charter, requirements, scope definition, scheduling, issue creation
2.0	System Design	Architecture, DFDs, UI design, database design, API planning
3.0	Frontend Development	Authentication UI, dashboard, Canvas views, forms, assistant UI, previews
4.0	Backend Development	Authentication APIs, Canvas services, workflow APIs, validation, persistence
5.0	Canvas Integration	Courses, students, assignments, submissions, messaging, grades, quizzes
6.0	AI/LLM Integration	Natural-language interpretation, provider integration, structured responses
7.0	Safety and Validation	Dry-run preview, confirmation, risk classification, error handling
8.0	Administration	Instructor management, audit logs, health information, failed operations
9.0	Testing and QA	Unit, integration, system, usability, acceptance, end-to-end testing
10.0	DevOps and Deployment	Docker, Docker Compose, CI/CD, environment setup
11.0	Documentation	Design documents, testing reports, user guides, developer guides
12.0	Final Delivery	Final report, presentation, validation, handover package

The WBS reflects the actual final architecture and feature set rather than only the initial UI. It captures both visible product functionality and supporting engineering activities such as testing, documentation, CI/CD and deployment.

Figure 6.1 - Work Breakdown Structure is shown on the following page.

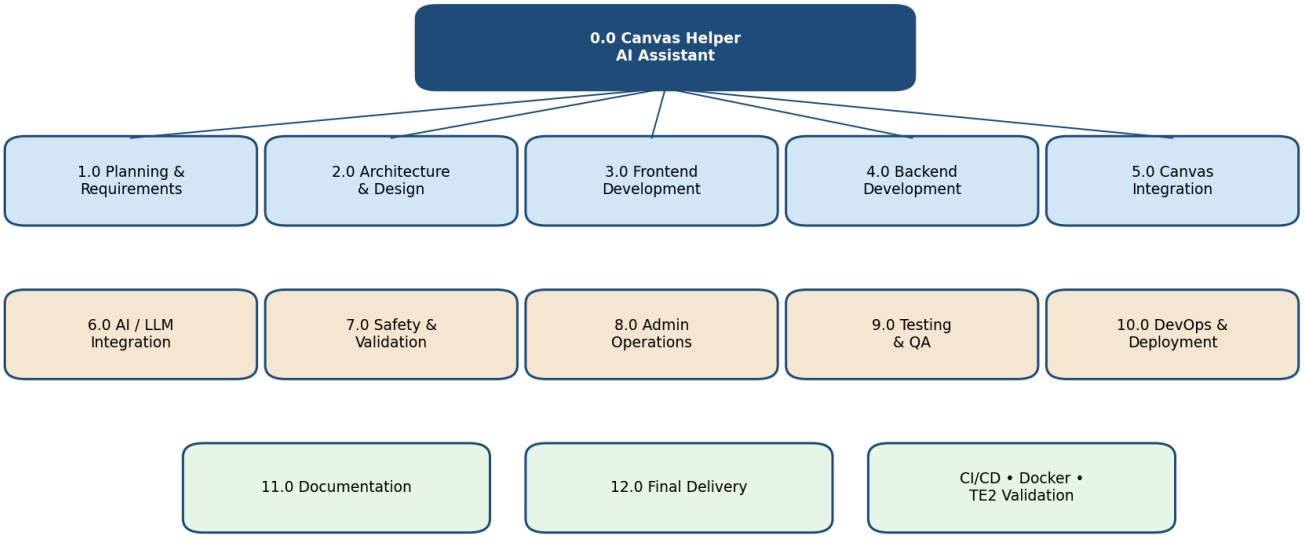
6.4 Feature Breakdown Structure (FBS)

While the WBS focuses on **work performed**, the Feature Breakdown Structure focuses on **capabilities delivered to users**.

Level 1 – Canvas Helper AI Assistant

FBS ID	Feature Group	Delivered Features
F1	Authentication and Access	Login, logout, session handling, RBAC
F2	Administrator Management	Create/update/enable/disable/revoke instructor accounts

Figure 6.1 – Work Breakdown Structure



Supporting packages span testing, documentation, deployment, and handover.

Source: project WBS in Section 6.3 of the Final Design and Analysis Report.

Figure 6.1 - Work Breakdown Structure. Hierarchical work packages spanning planning through final delivery.

F3	Canvas Connection	Secure token setup, validation, replacement/removal
F4	Course Dashboard	Courses, active course, students, assignments, submissions
F5	Submission Analysis	Missing, late, submitted and graded submission status
F6	Natural-Language Assistant	Conversational requests, intent interpretation, clarification
F7	Assignment Management	Create and update assignments
F8	Quiz Management	Create and validate quizzes
F9	Messaging	Draft, edit, preview and send Canvas messages
F10	Feedback	Draft, review and post feedback comments
F11	Grade Management	Preview and execute approved grade modifications
F12	Preview and Confirmation	Dry-run preview, cancellation, risk classification
F13	History and Reversal	Action history, assistant history, limited reversal
F14	Admin Operations	Audit logs, system health, failed-operation monitoring
F15	Security and Reliability	Encryption, credential isolation, safe errors, duplicate protection
F16	Deployment and Support	Docker deployment, CI/CD, developer and user documentation

These feature groups correspond to the final functional areas verified during Testing Event 2, where all listed requirements achieved PASS status.

Figure 6.2 - Feature Breakdown Structure is shown on the following page.

6.5 Team Work Distribution

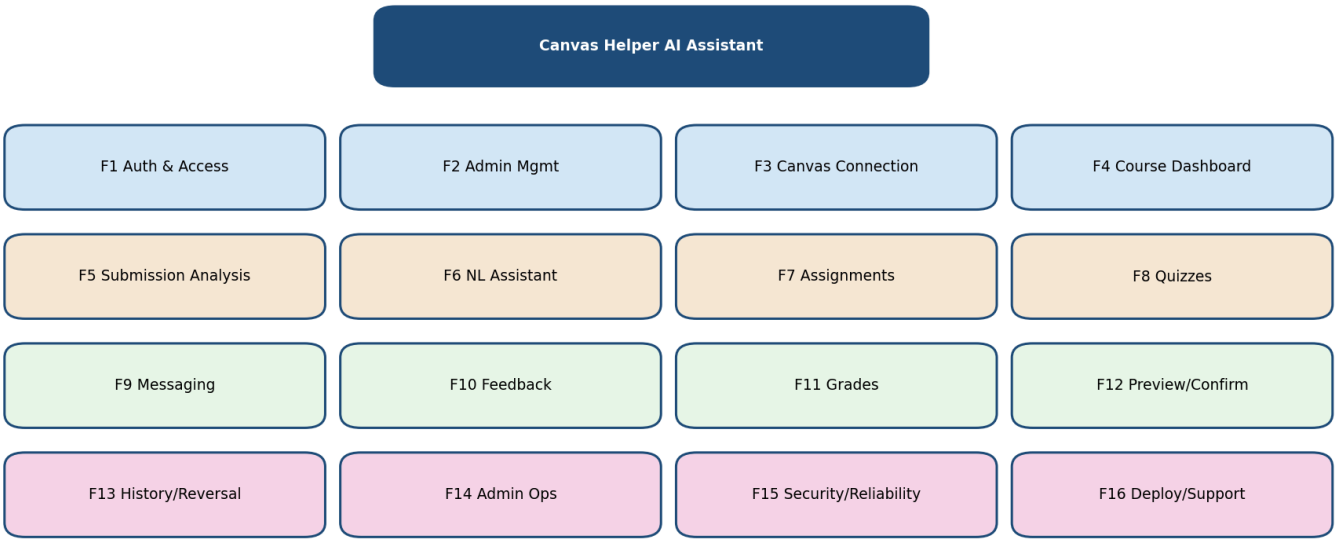
The project plan intentionally distributed responsibilities across multiple technical areas rather than assigning each person to only one layer.

Work Area	Mridul	Bao	Jessica	Yerdana	Rhythm
Backend Development	□	□	□	□	
Frontend Development		□	□		□
AI/LLM Integration	□		□	□	
DevOps / Docker	□		□	□	□
Documentation	□	□	□	□	□
Testing / QA	□	□	□	□	□
Presentations	□	□	□	□	□

This responsibility matrix was defined in the project plan.

Actual implementation later involved additional overlap between members, as demonstrated by the contributor mappings in Testing Event 2. For example, Jessica contributed to authentication, Canvas data access, natural-language workflows, assignment/quiz management, messaging, feedback, grading, confirmation, AI validation, security, reliability and documentation/deployment requirements.

Figure 6.2 – Feature Breakdown Structure



All feature groups map to TE2 requirement areas recorded as PASS.

Figure 6.2 - Feature Breakdown Structure. Feature groups F1-F16 correspond to capabilities verified as PASS in Testing Event 2.

This cross-functional distribution reduced bottlenecks and allowed team members to contribute to integration work rather than remaining isolated within frontend or backend responsibilities.

6.6 Time Tracking and Weekly Effort

Individual and team effort was tracked throughout development. The repository defines the weekly-log structure as containing both personal and team logs, with individual logs organized by date and separate team logs expected for each week.

Week / Cycle	Jessica Est.	Jessica Actual	Mridul Est.	Mridul Actual	Bao Est.	Bao Actual	Yerdana Est.	Yerdana Actual	Rhythm Est.	Rhythm Actual
Week 1	18	8	18	4	18	1	18	0	18	0
Week 2	18	10	18	7	18	9	18	5	18	10
Week 3	18	18	18	16	18	18	18	10	18	17
Week 4	18	17	18	15	18	20	18	12	18	12
Week 5	18	12	18	10	18	15	18	11	18	9
Week 6	18	14	18	8	18	25	18	15	18	15
Week 7	18	17	18	23	18	18	18	28	18	16
Week 8	18	18	18	31	18	24	18	54	18	21
Week 9	18	19	18	18	18	15	18	19	18	16
Week 10	18	21	18	45	18	21	18	26	18	22
Week 11	18	23	18	30	18	16	18	35	18	14
Final Week	18	34	18	28	18	12	18	36	18	28
Total	**216**	**211**	**216**	**235**	**216**	**194**	**216**	**251**	**216**	**180**

Team Member	Estimated Total Hours	Actual Total Hours	Variance
Jessica Goel	216	211	−5
Mridul Nohria	216	235	+19
Bao Tran	216	194	-22
Yerdana Maulenbay	216	251	+35
Rhythm Trivedi	216	180	-36
Total Project Effort	**1080**	**1071**	**−9**

6.7 Estimate versus Actual Analysis

The comparison between estimated and actual effort should be used to identify where the team's original planning assumptions were accurate and where the complexity of implementation changed the workload.

Throughout the project, actual effort varied from the initial estimates because the complexity of individual work packages became clearer during implementation. Integration-heavy tasks generally required more effort than isolated interface or service development because they involved coordinating frontend behaviour, backend contracts, Canvas API behaviour, AI output validation and testing. Additional effort was also required after user testing when existing workflows were refined rather than merely adding new features.

The team responded to estimation variance by updating GitHub issues, breaking larger features into smaller work packages, redistributing tasks where necessary and prioritizing requirements according to upcoming project milestones. This iterative estimation process provided more accurate short-term planning than attempting to predict the entire implementation workload at the beginning of the project.

Concrete examples from the tracked hours illustrate these variances. Week 8 was estimated at 90 team hours but required 148 hours, primarily because Canvas messaging, quiz validation, and integration debugging overlapped in the same cycle. Week 10 was estimated at 90 hours but required 135 hours, primarily because Testing Event preparation and late-cycle requirement re-validation expanded the workload beyond the original plan.

6.8 Burnup / Burndown Tracking

The team used milestone and issue tracking to monitor implementation progress throughout development. GitHub Projects was identified in the project plan as the Kanban and milestone-tracking mechanism used to coordinate the team.

Progress was monitored through GitHub milestones and issue completion. Figure 6.3 - Final Project Burnup Chart appears on the following page. The interpretation below summarizes the trend shown in that chart.

Figure 6.3 shows the GitHub Projects burnup of tracked issues from May through August. Completed work rose steadily with the growing scope, including a mid-June scope increase, and by final delivery the completed line nearly meets the total open scope. Remaining work at freeze was a small gap between open and completed items.

Where planned and completed work diverge on the chart, the gap reflects scope added after user testing and client feedback rather than incomplete delivery of the original core requirements.

6.9 Features and Work Packages Completed

By final validation, work packages covered all major functional areas of the application.

Testing Event 2 shows complete PASS coverage for authentication, Canvas connection, dashboard/data access, natural-language interaction, assignment and quiz management, submission analysis, grading, messaging and feedback, preview/confirmation, history/reversal, AI validation, security/privacy, reliability and deployment requirements.

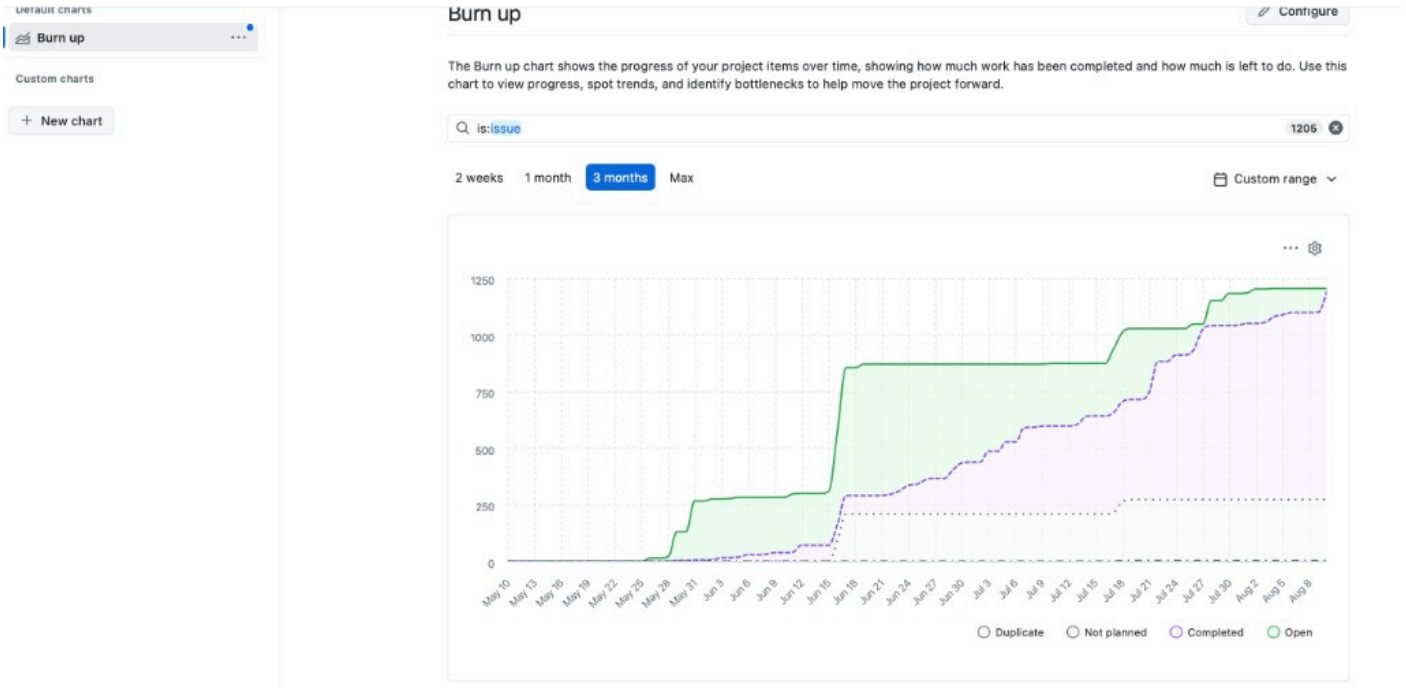


Figure 6.3 - Final Project Burnup Chart (GitHub Projects). Open scope (green) versus completed work (purple) from May through August; nearly all tracked issues were completed by final delivery.

From a project-management perspective, this means the team completed not only user-facing feature packages but also supporting work such as:

- architecture and system design;
- automated and manual testing;
- integration and regression work;
- deployment configuration;
- CI/CD;
- final technical documentation;
- user documentation; and
- submission and handover preparation.

The final repository package itself contains dedicated user guides, developer documentation, deployment validation, architecture/API references, testing documentation, requirements traceability and final submission materials.

6.10 Project Management Challenges and Adjustments

Several characteristics of the project created management challenges.

The original project plan identified LLM output reliability, frontend-backend integration, team coordination and context-window limitations as anticipated hurdles. The planned mitigations included structured validation, previews, early API specifications, mock responses, GitHub Projects, weekly sync meetings, PR reviews and milestone tracking.

During development, the project also evolved from its initial design. Some originally planned boundaries were expanded as later features such as grade modification and limited reversal were implemented with appropriate human confirmation and validation. This required careful scope management because new functionality still had to preserve the project's safety requirements.

The team managed these changes by decomposing work into smaller GitHub issues, assigning ownership, using pull requests for integration, and validating completed functionality during formal testing events. This prevented the evolving system from becoming disconnected from its requirements.

6.11 Final Project Management Assessment

Overall, the project-management process supported the successful delivery of a technically broad capstone system across five team members.

The use of role assignments, GitHub issue tracking, milestone planning, code review, time tracking, iterative testing and documentation enabled parallel development while maintaining a common architecture. The original project plan established the team's responsibility areas and coordination mechanisms, while the final system demonstrates substantial cross-functional contributions and integration across those roles.

The final Testing Event 2 results provide a useful project-management outcome measure: all documented requirements were recorded as PASS, indicating that the team's planning and implementation activities ultimately produced the required functionality.

7. Lessons Learned and Project Reflections

7.1 Team Reflection

The development of the Canvas Helper AI Assistant provided the team with experience managing a complex software project involving frontend and backend development, Canvas API integration, artificial intelligence, security, testing, deployment, and user experience design. The project evolved significantly from the initial proposal as implementation challenges, client feedback, and user testing provided a clearer understanding of what was required to produce a usable and reliable system.

The team followed an iterative development process in which functionality was divided into smaller features and GitHub issues, implemented through individual branches, reviewed through pull requests, integrated, and subsequently tested. The original project plan identified GitHub Projects, weekly meetings, pull-request reviews, and milestone tracking as important coordination mechanisms. As development progressed, responsibilities increasingly crossed the original frontend/backend boundaries because complete features required integration between several components.

This was reflected in the work completed across the project. Team members had to contribute not only to isolated code tasks, but also to planning, design decisions, service orchestration, testing, documentation, and bug fixing. For example, work on features such as chat history reload, course dashboard analytics, message sending, AI assistant window behavior, Dockerized services, CI workflows, Makefile setup, and database schema design required coordination across different parts of the system. These tasks showed that a feature was not complete simply because one component worked; it also had to integrate correctly with the frontend, backend, database, Canvas data, and user workflow.

One of the most important lessons from the project was that building an AI-enabled application requires more than successfully connecting an LLM to an interface. AI-generated actions needed validation, meaningful error handling, appropriate context, and safeguards before they could be trusted within an educational system. This resulted in the final confirm-before-write architecture, where Canvas-modifying operations are validated and presented to the instructor before execution.

Additionally, designing tools, and creating workflows for the tools turned out to be the most challenging and the most vital part that enabled AI Assistant functionality. Over the development period, team members listened to feedback from others, that helped to redefine user workflows.

User testing also demonstrated the importance of iterative UX development. Feedback concerning technical confirmation messages, action risk, editable AI output, persistent course context, and error handling resulted in changes to the final application. The project therefore reinforced the importance of treating testing and user feedback as part of development rather than activities performed only after implementation.

The project also strengthened non-technical skills across the team. Working under milestone pressure required better time management, communication, issue tracking, and coordination between dependent tasks. Testing events, final bug fixes, PR reviews, and final delivery preparation showed the importance of staying organized when several parts of the system were changing at the same time.

By final validation, all documented user and functional requirements evaluated during Testing Event 2 achieved PASS results. The final outcome demonstrates the team's ability to adapt its original design, coordinate work across several technical areas, and bring the individual components together into a complete system.

7.2 Mridul Nohria – Individual Reflection

The Canvas Helper AI Assistant was valuable to me because my work slowly became much bigger than just frontend code. At the start, I was mainly working on planning, frontend tasks, and project setup, but later I worked across backend integration, Canvas workflows, AI features, testing, PR reviews, documentation, and final project preparation. I also helped with important setup work like the initial ER schema, GitHub CI workflow, Makefile, and Dockerization of the frontend, backend, Canvas, and databases. This helped me understand how important project structure and consistent setup are when a team is working on the same system.

I worked on features and fixes such as chat history reload, analytics on the course dashboard, message sending, Canvas URL integration, AI window bugs, scroll bugs in the AI agent, grade management, action reversal, and risk-based confirmation workflows. These tasks taught me that most features are not only frontend or backend. A feature only really works when the UI, backend API, database, Canvas data, validation, and testing all work together.

One of the biggest things I learned was feature ownership. It is not enough to only finish the code. The feature also needs to be tested, reviewed, integrated, fixed, and documented. A lot of bugs only showed up during peer testing and Testing Event 2, so I learned that integration and user testing should happen earlier, not only near the end.

I also learned a lot about AI-assisted design. AI features need clear limits, editable previews, confirmation steps, and user control. More information is not always better. For example, confirmation screens should show enough detail for the user to understand the risk, but not so much that the decision becomes confusing.

Beyond technical skills, I learned planning, teamwork, communication, and time management. Working with GitHub issues, PRs, CI, project boards, testing feedback, and deadlines helped me understand how real software work depends on organization as much as coding. I also had to manage pressure during exams, testing events, bug fixes, and final delivery.

Overall, this project helped me grow as a full-stack developer and as a teammate. I learned that building software is not just about writing code. It is also about design, planning, testing, documentation, communication, and being able to adapt when the project becomes stressful or complicated.

7.3 Jessica Goel – Individual Reflection

The Canvas Helper AI Assistant was particularly valuable to me because my responsibilities expanded significantly throughout the project. I initially contributed heavily to frontend development and project planning, but as the system evolved I also worked across backend development, integration, AI-assisted workflows, testing, documentation, and final feature completion. This required me to understand not only how individual interfaces worked but also how they interacted with backend APIs, Canvas data, validation, and the AI workflow.

One of the biggest lessons I learned was the importance of taking ownership of a feature from implementation through integration and testing. Features such as assignment and quiz management, messaging, feedback, grading, authentication, and confirmation workflows were not purely frontend tasks. Completing them required understanding the full flow from the user interface through backend processing and ultimately to Canvas. The final testing report reflects my contributions across authentication, Canvas data access, natural-language functionality, assignment and quiz workflows, messaging and feedback, grading, confirmation, AI validation, security, reliability, and deployment-related requirements.

Another important lesson came from user feedback. Initially, I tended to think that providing users with more information would automatically make an interface clearer. Testing showed that this was not always true. Confirmation screens containing too much technical information could make decisions harder rather than easier. Similarly, AI-generated content still needed to remain editable because instructors must retain control over what is ultimately sent to Canvas. This changed how I think about UX: a good interface does not simply expose information; it presents the right information at the right time.

The project also improved my ability to work within a team using GitHub issues, branches, pull requests, reviews, CI, and iterative development. There were periods when dependencies between features and integration work made planning difficult, but these challenges taught me to break larger problems into smaller tasks and communicate dependencies earlier.

In future projects, I would begin end-to-end integration and user testing earlier instead of treating them primarily as later development activities. Overall, the project strengthened my frontend and full-stack development skills, improved my understanding of AI-assisted application design, and taught me to think about security, usability, testing, and maintainability as part of implementation rather than separate concerns.

7.4 Bao Tran – Individual Reflection

Most of my time on the Canvas Helper AI Assistant went into the backend, starting from an empty folder in Cycle 5. I set up the initial FastAPI project, the Postgres schema, and the integration, service, and router structure that everything else got built on. From there it was auth and JWT, role-based access, per-user Canvas token encryption, and the Canvas retrieval services for courses, students, assignments, and submissions. Later I was assigned three features for Testing Event 2: System Health Monitoring, Audit Logging, and Secure Troubleshooting. Owning the backend the whole way through, instead of one feature at a time, meant I could always see how the pieces fit together, which is also why I kept running into design problems that only showed up once real endpoints depended on the schema, things like the co-instructor uniqueness rule, the active-course table, and scoping Canvas

sync to the right school for instructors teaching at more than one institution. None of that was planned up front, it just came up as features landed on top of each other.

The gaps that actually cost me time were not technical, they were the moments I built backend against an assumption instead of checking it with whoever owned the frontend side. Early on I had the register form and the backend running in parallel with a teammate, and neither of us noticed the signup form wasn't calling the backend yet and the name field didn't even exist on that side until it was already a problem. A few cycles later the same thing happened with the Canvas token endpoints, edge cases in validation took longer than they should have because I was guessing at what the frontend actually needed instead of asking first. Both times I wrote in my own retro that a quick alignment conversation up front would have saved the rework, and both times I still let it happen again before it actually stuck.

A feature being done never actually meant the code compiled. JWT role checks that were not being re-verified against the database, an audit log silently dropping rows past 5000 during pagination, an error field reaching an admin's screen with a credential still inside it. I caught all of these from review or from actually running the thing end to end, not from writing the code faster. I started re-listing carried-over goals every cycle instead of letting them quietly drop, which is the only reason a Canvas URL security check a teammate's PR had weakened stayed on my radar for three cycles straight, even though I still have not landed the actual fix.

If I am honest about what I would do differently, it is that I stayed too much on the guardrails around the AI and not enough inside it. I built the validation and safety layer that sits around the LLM, checking IDs against the real database, redacting secrets, gating risky actions behind confirmation, but I never really dug into how the model was actually being prompted or why it produced the outputs it did. I understood the assistant from the outside, as something to contain, more than from the inside, as something to actually reason about. That is a gap I want to close, because right now I could not confidently explain why a prompt change would help or hurt without testing it first.

The other thing I would put more effort into is the UI on the pages tied to my own features. The System Health, Audit Logging, and Troubleshooting pages worked and were tested, but they were built to expose the data correctly, not to be genuinely easy or pleasant for an admin to actually use. Compared to teammates who spent real time on layout and interaction details, my frontend work was mostly functional patchwork on top of backend logic. An admin-facing tool is still a tool someone has to read under pressure when something is actually broken, so a clearer, less cluttered UI there would have mattered as much as the backend correctness I was focused on.

Overall, this project moved me from someone who writes an endpoint to someone who owns a backend other people's features depend on, and that part I am confident in. What I am taking into the next project is that owning the backend well is not the same as understanding the whole system well, and that most of what actually broke came from not talking to the right person early enough, not from the code itself. I also want to spend real time inside the AI logic next time, not just around it, and treat the UI and the frontend conversation as part of the feature instead of the last thing I get to if there is time left.

7.5 Yerdana Maulenbay – Individual Reflection

Working on AI Assistant for Canvas with the team was incredibly rewarding, challenging, and served as a great learning experience. Despite being a team of 5, our team was incredibly organized with deadlines, meeting expectations from teaching staff POV, and with delivering a working product. At the beginning, I needed some time to adapt to the pace of the work, as well as the amount of deliverables that I had to produce, however, Jessica and Mridul were very adept at planning which certainly made it easier.

By the time I started developing our software, I had one clear goal in mind which was to focus predominantly on being technically proficient. While this approach hasn't failed me, it highlighted the importance of effective communication, having a common vision, and premature testing more than anything. Initially, I was responsible for the backend and while I helped the team to set up a solid foundation on which to build on, I would coordinate with the frontend team more often. Actually I would do so using simple language and even bypassing long PR descriptions on Github as I saw how impactful an alignment of our ideas can be. I single-handedly felt this while testing, where I had no idea what I was trying to test without reading between the lines of PR descriptions. The next most impactful change I wish I embraced sooner would be testing, which I may have made fun of when Dr. Fazackerley mentioned it more than anything during lectures. Now I take my words back, and the reason for that is testing helps tremendously with planning new features. One such hurdle along our way was integration testing right before the testing event 1, which not only tested our features but also our patience to some extent. Subsequently as testing was taken more seriously, we decided to split the tasks differently, where each person is assigned a feature and gets full ownership. I was responsible for testing, and integration which I found more engaging than developing a feature since it allowed me to have a big picture in my head.

As the saying hindsight is 2020 goes, I genuinely appreciate the learnings from our highs and lows as the team. Without learning what we should avoid, or do better, I wouldn't have the skill set, the experience, and appreciation for other applications that use external LLM. After all the work I have put into this project, I genuinely treat Canvas AI Assistant not as an LLM wrapper but rather as a standalone app.

7.6 Rhythm Trivedi – Individual Reflection

Working on Team 16's Canvas Workflow Assistant pushed me out of a frontend-only comfort zone. I began with instructor screens such as course lists, dashboards, analytics, and submission views, but most of the value later came from work that crossed boundaries: chat and AI UX, auth feedback, content safety, Canvas course creation, admin failed-ops monitoring, LLM provider settings, local seeding and Makefile tooling, testing reports, and design-doc updates. I stopped thinking in terms of isolated components and started thinking in terms of full instructor paths that have to survive Canvas quirks, validation, and CI.

A recurring theme for me was unfinished integration. It is easy to land UI that looks right while the handoff to the backend or Canvas still fails in edge cases. Owning pieces such as password-policy feedback, assignment clarification in chat, analytics replies for missing work, AI loading states, starter prompts, profanity and safety checks, and course publish flows taught me

to verify the whole chain before calling something done. The same lesson applied to developer experience. Aligning `make test` with CI and seeding Canvas students for local demos mattered because the team could not evaluate the product without a reliable sandbox.

I also changed how I judge good AI assistance. Early instincts leaned toward showing more model output and more options. In practice, instructors needed clearer progress while waiting, fewer dead ends when fields were missing, and guardrails that did not punish legitimate academic wording. Safety without usable recovery, or clever chat without editable, approvable results, does not help teaching staff. That tension between helpful automation and instructor control shaped a lot of my later bugfix and UX work.

Collaboration-wise, the project trained me on issue-driven development, review culture, and how quickly parallel merges can interact. Writing weekly logs and contributing testing-event documentation also made me more deliberate about recording what changed and why.

If I repeated a project like this, I would schedule real Canvas walkthroughs earlier and keep fewer long-lived branches that touch the same chat and course surfaces. I leave the course more confident as a full-stack contributor: stronger on React and TypeScript delivery, more comfortable in FastAPI and AI workflow code, and clearer that security, testing, and documentation are part of building the feature, not paperwork afterward.

7.7 Overall Lessons Learned

Across the individual reflections and the team's shared experience, several project-wide lessons stand out.

1. Human-in-the-loop is a product requirement, not only a safety slogan.
Confirm-before-write, editable drafts, and clear risk/reversibility cues were essential for instructor trust. AI output that could not be reviewed, corrected, or cancelled was not useful in a Canvas teaching context.

2. Integration cost dominates isolated feature cost.
Work that looked small in a single layer - a chat reply, a Canvas write, a preview dialog - became expensive once frontend state, backend validation, Canvas behaviour, LLM variability, and tests had to agree. Estimation improved only after the team planned around end-to-end workflows rather than individual screens or endpoints.

3. Course and conversation context must be explicit.
Many usability and correctness issues traced back to ambiguous active-course state or stale workflow context. Persisting course selection, clarifying when the assistant switches context, and validating write targets against the intended course were recurring fixes.

4. Testing and developer experience are part of delivery.
Automated Pytest/Vitest/Playwright coverage, CI alignment with local make test, and a usable Canvas sandbox were necessary to merge confidently and to demonstrate the system. Late discovery of environment friction slowed otherwise-complete features.

5. Scope evolves; architecture must absorb change safely.
Features such as grade modification, messaging delivery modes, and limited reversal expanded beyond the earliest design. The team managed this by keeping Canvas as the source of truth, requiring backend validation, and refusing silent AI-side mutations.

6. Collaboration quality determines integration quality.
Issue-driven work, pull-request review, and frequent sync reduced - but did not eliminate - overlap on shared surfaces such as chat and messaging. Shorter-lived branches and earlier joint walkthroughs on real Canvas data would have reduced late merge conflict and duplicated fixes.

7. Documentation and hand-over are engineering outputs.
Traceability, deployment guides, user workflows, and testing evidence were required both for evaluation and for any future maintainer. Treating them as last-minute paperwork would have left a working demo without a transferable product.

Overall, the project succeeded when the team optimized for instructor control, end-to-end correctness, and maintainable integration - not for maximizing autonomous AI behaviour. Those priorities should guide any future extension of the Canvas Helper AI Assistant.

8. Hand-over, Installation, Setup and User Documentation

8.1 Purpose

This section provides the information required for another developer, administrator, or evaluator to install, configure, run, test, maintain, and continue development of the Canvas Helper AI Assistant after project hand-over.

The final repository includes dedicated documentation for deployment, development, architecture, API usage, testing, requirements traceability, and user guidance. The application is designed as a web-based system with a React frontend, FastAPI backend, PostgreSQL database, Canvas integration, LLM-provider integration, and an optional MCP/tool boundary.

The hand-over process should therefore provide the recipient with:

- access to the source repository;
- installation and dependency information;
- environment configuration instructions;
- deployment instructions;
- testing procedures;
- user documentation;
- development workflow information; and

- troubleshooting guidance.

8.2 Repository and Project Structure

The final project source code and documentation are maintained in the team's GitHub repository.

The final submission package contains dedicated documentation covering architecture, API information, deployment, developer guidance, user guidance, testing, requirements traceability, and final release information.

At a high level, the system is organized into the following components:

Component	Responsibility
Frontend	React-based user interface
Backend	FastAPI application services and API
Database	PostgreSQL persistent storage
Canvas Integration	Controlled Canvas LMS API communication
AI Integration	LLM-provider communication and AI-assisted workflows
MCP/Tool Layer	Optional controlled tool/credential boundary
Testing	Backend, frontend and end-to-end tests
Deployment	Docker/Docker Compose configuration
Documentation	User, developer, deployment and technical documentation

The separation of these components allows future developers to modify one area of the application while maintaining clear boundaries with the remainder of the system.

8.3 Development and Deployment Environments

The project supports both a development/testing environment and a complete deployment environment.

The repository's quick-start documentation provides Docker-based commands for bringing up the required services. In development, PostgreSQL and the Canvas sandbox can be started together, followed by the API and frontend. The project also provides a full-stack deployment configuration using nginx.

Development Environment

The development environment is intended for implementing features, debugging, running automated tests, and testing integrations.

It consists of:

Frontend

- React application
- frontend development server
- Vitest testing environment

Backend

- FastAPI
- Python runtime
- Pytest testing environment

Database

- PostgreSQL

External/Supporting Services

- Canvas or Canvas sandbox
- configured LLM provider
- optional MCP/tool service

Deployment Environment

The deployment environment runs the application's required services using Docker/Docker Compose. The final repository also provides full-stack deployment support with nginx.

Docker reduces environment differences by packaging the required application components and dependencies into reproducible services.

8.4 Installation Prerequisites

Before installing the project, the developer should ensure that the required development tools are available.

At minimum, the environment must support:

- Git;
- Docker;
- Docker Compose;
- the application's frontend development environment;
- Python/backend dependencies where running outside containers;
- network access to the required Canvas environment; and
- access to a configured LLM provider where AI functionality is being tested.

The exact runtime and dependency versions should be taken from the final repository manifests and lockfiles rather than manually choosing newer versions. This is important because upgrading dependencies during hand-over may introduce compatibility problems.

> **Hand-over rule:** use the versions committed with the final release whenever possible before attempting dependency upgrades.

8.5 Initial Installation

The recommended installation process begins by obtaining the final repository.

Step 1 – Clone the Repository

Clone the final version of the project and move into the project directory.

git clone <repository-url>
cd capstone-team-16

The hand-over recipient should use the final release/submission branch or tag rather than an older development branch.

Step 2 – Review Environment Configuration

Before starting the application, review the environment configuration expected by the repository.

Environment values should be configured according to the final deployment documentation.

Sensitive values must **never** be committed directly to the repository.

Before production deployment, the customer must provide unique secure values for CANVAS_TOKEN_ENCRYPTION_KEY, SECRET_KEY, and—if MCP is enabled—MCP_SHARED_SECRET

These may include:

- database configuration;
- authentication configuration;
- Canvas-related configuration;
- encryption secrets;
- AI-provider configuration; and
- deployment-specific values.

Canvas and AI credentials are treated as sensitive information by the application's security model. The requirements specifically prohibit sensitive credentials from appearing in source repositories, logs, errors, API responses, or UI output.

8.6 Running the Development Environment

The final repository provides Make commands to simplify common development operations.

The documented quick-start process can start the supporting PostgreSQL and Canvas sandbox services and then run the API and UI development environment.

Use the exact commands in the final repository README/Makefile when performing hand-over, because they represent the tested final configuration.

Once the services are running, verify that:

1. the database service is healthy;
2. the backend starts without configuration errors;
3. the frontend can communicate with the backend;
4. the login interface loads;
5. Canvas connectivity can be configured;
6. the selected LLM provider is available; and
7. modifying workflows display a preview before execution.

8.7 Running the Full Application with Docker

Docker Compose provides the preferred reproducible deployment approach for the complete system.

The final project includes Docker-based deployment support, and TE2 verified Docker Compose deployment as a passing requirement.

The repository also documents a full-stack deployment configuration using nginx.

The recommended deployment sequence is:

Before production deployment, the customer must provide unique secure values for CANVAS_TOKEN_ENCRYPTION_KEY, SECRET_KEY, and—if MCP is enabled—MCP_SHARED_SECRET

Environment Configuration → Docker Build → Start Services → Verify Database → Verify Backend → Verify Frontend → Configure Canvas → Configure AI → Run Smoke Tests

After deployment, the administrator should verify that protected functionality cannot be accessed without authentication and that Canvas-changing operations require explicit confirmation.

8.8 Testing the Installation

A new developer should run the automated test suite immediately after installation. This verifies that the environment matches the configuration used by the development team.

The final repository provides three primary testing commands:

make test
make test-e2e
make test-fuzz

make test runs backend **Pytest** and frontend **Vitest** coverage. make test-e2e runs the **Playwright** end-to-end test suite. make test-fuzz runs the optional **Hypothesis** fuzz-testing suite.

The recommended order is:

1. Run make test.
2. Resolve any environment or dependency failures.
3. Start the complete application.
4. Run make test-e2e.
5. Optionally run make test-fuzz.
6. Perform a manual smoke test of the main Canvas workflows.

Testing Event 2 provides the final requirement-level baseline against which future versions can be compared. All requirements represented in that report achieved PASS status.

8.9 Manual Smoke-Test Procedure

After automated testing succeeds, the hand-over recipient should manually verify the most important user workflows.loca

A recommended smoke test is:

1. Open the application.
2. Authenticate using a valid account.
3. Verify that invalid or disabled accounts are rejected.
4. Connect a valid Canvas account/token.
5. Retrieve available courses.
6. Select an active course.
7. Retrieve assignments and submissions.
8. Submit a read-only request through the Natural-Language Assistant.
9. Create a proposed assignment or quiz.
10. Verify that a preview appears before execution.
11. Cancel the proposed action and verify that Canvas is not modified.
12. Repeat the operation and explicitly approve it.
13. Verify that the approved change reaches Canvas.
14. Review the resulting action history.
15. Test a messaging or feedback workflow.
16. Test administrator account-management functionality.
17. Verify audit/health information.
18. Log out and confirm that protected functionality can no longer be accessed.

This smoke test validates the application's main end-to-end path and, importantly, verifies that the confirm-before-write safety mechanism remains operational.

8.10 User Manual – Instructor

8.10.1 Logging In

The instructor begins by opening the web application and entering their application credentials.

After successful authentication, the instructor is provided access to the instructor-facing dashboard and Canvas functionality.

If authentication fails, the application displays an appropriate error. Disabled or revoked accounts cannot access protected functionality.

8.10.2 Connecting Canvas

Before Canvas functionality can be used, the instructor must establish a valid Canvas connection.

The instructor provides the required Canvas credential through the application's Canvas connection interface.

The backend validates the credential before allowing Canvas-dependent functionality. Credentials are stored securely and are not displayed again after submission.

The instructor can later replace or remove the credential if necessary.

8.10.3 Selecting a Course

Once Canvas is connected, the instructor can retrieve available courses and select an **active course**.

The active course becomes the default context for dashboard and assistant workflows unless another course is explicitly selected.

This prevents ambiguous requests from unintentionally targeting the wrong course.

8.10.4 Using the Dashboard

The dashboard provides access to course-level information including:

- students;
- assignments;
- submissions;
- submission status;
- missing/late work; and
- workflow shortcuts.

The dashboard serves as the central location from which instructors can begin common Canvas-management tasks.

8.10.5 Using the Natural-Language Assistant

The instructor can enter conversational requests through the assistant.

Examples include requests to:

- list assignments;
- identify missing submissions;
- inspect students or submissions;
- create an assignment;
- update assignment information;
- create a quiz;
- draft a student message;
- generate feedback; or
- prepare a grade modification.

The assistant interprets the request and uses the current Canvas context where appropriate.

If a request is ambiguous, the application may require clarification before proceeding.

8.11 Creating or Updating Assignments

To create an assignment:

1. Select the appropriate course.
2. Open the assignment workflow or request assignment creation through the assistant.
3. Enter or generate the assignment information.
4. Review/edit the structured information.
5. Continue to preview.
6. Review the proposed Canvas change.
7. Confirm to execute or cancel to stop the action.

Assignment updates follow the same principle. Existing assignment information is retrieved, modified, validated and presented for confirmation before Canvas is changed.

8.12 Creating a Quiz

Quiz creation follows the same human-in-the-loop design.

The instructor prepares the quiz information, reviews the generated or entered content, and proceeds to preview.

The quiz is created in Canvas only after explicit instructor confirmation.

Quiz creation and validation were verified as passing during TE2.

8.13 Messaging Students

The messaging workflow allows the instructor to generate Canvas message drafts.

The instructor can:

1. select the relevant student(s);
2. generate a message;
3. review the draft;
4. edit the message;
5. preview the proposed operation;
6. approve or cancel; and
7. send the approved message.

Messages are not automatically sent merely because the AI generated them.

8.14 Generating Feedback

The instructor can generate feedback for selected submissions.

Generated feedback remains reviewable before posting. The instructor can modify the feedback and explicitly approve it before it is submitted to Canvas.

This maintains instructor responsibility over student-facing communication.

8.15 Modifying Grades

Grade modification is treated as a high-impact workflow.

A proposed grade change is validated and shown to the instructor before execution.

The instructor must explicitly confirm the operation before the backend modifies the grade in Canvas.

This workflow should never be changed in future development to allow autonomous AI grade modification without human confirmation.

8.16 Understanding Preview and Confirmation

The preview screen is one of the application's primary safety controls.

Before a Canvas-changing operation, the instructor is shown information describing:

- the proposed action;
- affected Canvas resources;
- relevant changes;
- risk level; and
- whether reversal is supported.

The instructor can then **Confirm** or **Cancel**.

Selecting Cancel must leave Canvas unchanged.

The final architecture defines confirm-before-write as a core system requirement.

8.17 Action History and Reversal

The instructor can review previous application actions through history functionality.

History can include:

- original request;
- interpreted action;
- execution result;
- timestamp;
- affected Canvas resource; and
- reversibility information.

Some supported operations may be reversed. Reversal is intentionally limited and should not be assumed to be available for every Canvas action.

8.18 User Manual – Administrator

Administrators have responsibilities that differ from instructor workflows.

Administrative functionality includes:

- creating instructor accounts;
- updating instructor accounts;
- enabling/disabling accounts;
- revoking access;
- reviewing audit information;
- viewing operational health;
- reviewing failed Canvas/AI operations; and
- troubleshooting system problems.

Administrators should not require access to users' plaintext sensitive credentials to perform troubleshooting.

8.19 Troubleshooting

Application Does Not Start

Check:

- Docker is running;

- required services are available;
- environment configuration is present;
- database configuration is correct; and
- required ports are not already in use.

Review application/container logs for startup failures, but ensure sensitive credentials are never printed.

Canvas Connection Fails

Verify:

- the Canvas environment is reachable;
- the supplied Canvas credential is valid;
- the credential has not expired or been revoked; and
- The backend Canvas configuration is correct.

Do not print or expose the complete Canvas token while troubleshooting.

AI Functionality Fails

Verify:

- the configured LLM provider is available;
- required provider configuration is present;
- backend services can reach the provider; and
- the request is one of the supported workflows.

The application should fail safely rather than attempting an unvalidated Canvas operation.

Frontend Cannot Reach Backend

Check:

- backend service health;
- frontend API configuration;
- Docker networking;
- reverse-proxy configuration where applicable; and
- browser/network errors.

Tests Fail After Installation

First determine whether the failure is caused by:

- missing environment configuration;
- dependency/version differences;
- unavailable services;

- database initialization;
- test data; or
- an actual regression.

Avoid changing dependencies simply to make a failing test disappear without identifying the underlying cause.

8.20 Continuing Development

A future developer should follow the existing architecture rather than bypassing its safety boundaries.

When adding functionality:

1. Create or update the corresponding requirement.
2. Create a GitHub issue.
3. Identify affected frontend/backend components.
4. Implement backend validation.
5. Implement UI functionality.
6. Add automated tests.
7. Add preview/confirmation if Canvas will be modified.
8. Test integration with Canvas.
9. Update documentation.
10. Submit the work through the project's review process.

New Canvas-modifying functionality should **never** directly expose unrestricted Canvas API calls to the AI model.

The original scope explicitly excludes autonomous Canvas modification, backend-validation bypass, unrestricted deletion, institution-wide Canvas administration, and arbitrary user-defined Canvas API calls.

8.21 Hand-over Checklist

Before transferring responsibility for the project, the team should verify the following:

Hand-over Item	Required
Final source code available	□
Final branch/tag identified	□
README available	□
Architecture documentation available	□
API documentation available	□

Developer guide available	□
User guide available	□
Deployment guide available	□
Testing documentation available	□
Requirements traceability available	□
Environment configuration documented	□
Sensitive credentials removed from repository	□
Docker deployment validated	□
Automated tests runnable	□
Final requirement status documented	□

The final submission package already contains the major documentation categories required for this hand-over, including architecture, API, deployment, developer, user, testing and traceability documentation.

8.22 Hand-over Summary

The Canvas Helper AI Assistant has been structured so that another development team can install, test, operate, and extend the system using the final repository and accompanying documentation.

Docker-based deployment reduces environment-specific configuration differences, while automated Pytest, Vitest, Playwright and optional Hypothesis testing provide a baseline for detecting regressions.

Most importantly, future development should preserve the architectural safety principles established by the project: Canvas remains the source of truth, credentials remain protected, AI-generated operations undergo backend validation, and Canvas-modifying actions require explicit human confirmation.

9. Conclusion

The Canvas Helper AI Assistant was developed to reduce repetitive instructor workload in Canvas while preserving user control, safety, and transparency. The final system combines a React-based frontend, FastAPI backend, PostgreSQL database, Canvas LMS integration, LLM-provider support, and an optional MCP/tool boundary within a web-based architecture. A central design principle is that Canvas-modifying actions are never executed automatically by the AI; instead, they are validated, previewed, and require explicit user confirmation before execution.

Over the course of the project, the system evolved significantly from the initial design into a broader and more complete final implementation. The final feature set includes secure authentication and account management, Canvas connection and credential handling, course and submission access, missing and late submission analysis, a natural-language assistant, assignment and quiz management, student messaging, feedback workflows, grade modification, dry-run previews, action history, limited reversal, administrative health and audit functionality, AI validation, security controls, reliability safeguards, and Docker-based deployment. These features reflect the complete functional scope documented and tested during the project.

The project also demonstrated the importance of iterative development and user-centred refinement. Feedback from testing and client review influenced changes to confirmation workflows, course-context handling, editable AI output, and error messaging. These refinements improved the usability and safety of the final application while maintaining the project's human-in-the-loop philosophy.

Testing was performed using multiple approaches, including unit, integration, system functionality, usability, acceptance, operational acceptance, end-to-end, and optional fuzz testing. The final Testing Event 2 report records all listed user requirements, functional requirements, and scope constraints as PASS. The final repository also provides automated testing through Pytest, Vitest, Playwright, and Hypothesis-based fuzz testing.

From a project-management perspective, the team successfully coordinated work across frontend development, backend development, AI integration, Canvas integration, testing, DevOps, documentation, and user-experience refinement. The project plan distributed responsibilities across the team while supporting collaboration through GitHub Projects, pull requests, weekly coordination, and milestone tracking.

The final submission package also provides the documentation required for future maintenance and development. It includes architecture and API references, developer documentation, deployment and installation guidance, user documentation, testing artifacts, requirements traceability, and release information. This ensures that the project can be installed, evaluated, maintained, and extended beyond the capstone course.

Overall, the project achieved its goal of demonstrating a practical and responsible approach to AI-assisted course management. Rather than replacing instructor decision-making, the system uses AI to support instructors while preserving explicit human authority over consequential actions. The final result is a complete, tested, and documented application that combines conversational AI with Canvas workflows in a manner that prioritizes safety, transparency, usability, and maintainability.